Trabajo Biofísica: Clasificación y Correlación de las estadísticas en Pokémon

Velasco García, Lucía

Villaverde Turnes, David

Introducción

Pokémon es una saga de videojuegos creada y desarrollada por *Game Freak* en 1996, que se popularizó rápidamente debido a su estilo sencillo de juego y progresión. El objetivo de estos juegos radica en convertirse en entrenador de unas criaturas llamadas pokémon. Para poder mejorar y continuar en la historia principal deberás usar tus pokémon para combatir contra otros entrenadores y así convertirte en el mejor.

Estas criaturas están agrupadas en algo que llamamos tipos, que representan el elemento que los caracteriza. Estos tipos van desde afinidades más habituales como fuego, agua o electricidad, hasta otras más hetéreas, como fantasma, dragón, etc. Cada pokémon puede tener uno o dos tipos.

A mayores de esto, cada pokémon se caracteriza por tener unas determinadas estadísticas, que representan cuán fuertes son. Estas estadísticas son: puntos de salud, ataque, defensa, ataque, defensa especiales y velocidad.

Las estadísticas, en principio, no tienen por qué estar relacionadas con los tipos. Sin embargo, se cree que hay pares de tipos con estadísticas contrarias entre sí. Esto lo podemos observar, por ejemplo, en los tipos psíquico y lucha: los primeros se caracterizan por tener alto ataque y defensa (golpes 'de contacto') pero bajo ataque y defensa especial (ataques 'a distancia'), mientras que en los segundos ocurre justo lo contrario.

Nuestro objetivo en este trabajo será, en primer lugar, analizar la relación que puede haber entre las estadísticas de ataque-ataque especial y defensa-defensa especial de todos los pokémon. Después, escogeremos solo los que tengan tipo lucha o tipo psíquico y entrenaremos un modelo que sea capaz de predecir el tipo del pokémon a partir de sus estadísticas.

Como último ejercicio, vamos a utilizar otro tipo de clasificación entre todos los pokémon: si son o no legendarios. En los juegos, los pokémon legendarios se caracterizan por tener una fuerza y unas características extraordinarias. Vamos a comprobar si esto es verídico o no entrenando un modelo de red neuronal que analice todas las estadísticas de cada pokémon y que sea capaz de predecir si este es o no legendario.

Base de datos: https://www.kaggle.com/datasets/abcsds/pokemon?resource=download

---

Pasos previos

Vamos a, en primer lugar, cargar nuestra base de datos y echar un vistazo a qué hay: las distintas categorías que encontramos, si hay o no valores duplicados o perdidos, etc.

In [15]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Cargar los datos
df = pd.read_csv('Pokemon.csv')

# Información general
print('\n','=' * 50)
print(f"Total de Pokémon: {df.shape[0]}")
print(f"Total de variables: {df.shape[1]}")
print('\n','=' * 50)

print('\n','=' * 50)
print("\nColumnas del dataset:")
print(df.columns.tolist())
print('\n','=' * 50)

print('\n','=' * 50)
print("\nNúmero de Pokémon de cada tipo:")
print(df[['Type 1','Type 2']].stack().value_counts())
print('\n','=' * 50)

print('\n','=' * 50)
print(f'\n Número de Pokémon legendarios: {df["Legendary"].sum()}')
print(f'\n Número de Pokémon no legendarios: {df.shape[0] - df["Legendary"].sum(
print('\n','=' * 50)

display(df.head())
```

```
 ==================================================
Total de Pokémon: 800
Total de variables: 13

 ==================================================

 ==================================================

Columnas del dataset:
['#', 'Name', 'Type 1', 'Type 2', 'Total', 'HP', 'Attack', 'Defense', 'Sp. Atk',
'Sp. Def', 'Speed', 'Generation', 'Legendary']

 ==================================================

 ==================================================

Número de Pokémon de cada tipo:
Water       126
Normal      102
Flying      101
Grass        95
Psychic      90
Bug          72
Ground       67
Fire         64
Poison       62
Rock         58
Fighting     53
Dark         51
Dragon       50
Electric     50
Steel        49
Ghost        46
Fairy        40
Ice          38
Name: count, dtype: int64

 ==================================================

 ==================================================

 Número de Pokémon legendarios: 65

 Número de Pokémon no legendarios: 735

 ==================================================
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Genera
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	

Preparación y limpieza de los datos

Antes de entrenar la red neuronal, es necesario preparar el conjunto de datos para garantizar que la información utilizada sea adecuada.

En esta etapa se comprobarán las dimensiones del dataset, los tipos de variables, la existencia de duplicados y los posibles valores ausentes.

In [16]:
```
# Dimensiones iniciales
print('\n','=' * 50)
print("Dimensiones iniciales:", df.shape)
print('\n','=' * 50)

# Primeras filas
print('\n','=' * 50)
display(df.head())
print('\n','=' * 50)

# Información de las variables
print('\n','=' * 50)
df.info()
print('\n','=' * 50)

# Eliminación de duplicados
df = df.drop_duplicates()

print('\n','=' * 50)
print("\nDimensiones tras eliminar duplicados:", df.shape)
print('\n','=' * 50)

# Eliminación de valores perdidos
df = df.dropna()

print('\n','=' * 50)
print("\nDimensiones tras eliminar valores perdidos:", df.shape)
print('\n','=' * 50)
```

```
 ================================================
Dimensiones iniciales: (800, 13)

 ================================================

 ================================================
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Genera
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	

```
 ================================================

 ================================================
<class 'pandas.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 13 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   #           800 non-null    int64
 1   Name        800 non-null    str
 2   Type 1      800 non-null    str
 3   Type 2      414 non-null    str
 4   Total       800 non-null    int64
 5   HP          800 non-null    int64
 6   Attack      800 non-null    int64
 7   Defense     800 non-null    int64
 8   Sp. Atk     800 non-null    int64
 9   Sp. Def     800 non-null    int64
 10  Speed       800 non-null    int64
 11  Generation  800 non-null    int64
 12  Legendary   800 non-null    bool
dtypes: bool(1), int64(9), str(3)
memory usage: 75.9 KB

 ================================================

 ================================================

Dimensiones tras eliminar duplicados: (800, 13)

 ================================================

 ================================================

Dimensiones tras eliminar valores perdidos: (414, 13)

 ================================================
```

Vemos que se pierden algunas filas, es decir, algunos pokémon. Aproximadamente la mitad.

Como comentamos en la introducción, los pokémon pueden tener hasta dos tipos. Comprobaremos que lo que está ocurriendo es que el programa está eliminando los que solo tienen un tipo.

In [17]:
```
#Volvemos a cargar el archivo de origen
df = pd.read_csv('Pokemon.csv')

print('\n' + '=' * 50)
print(df['Type 2'].unique())
print('\n' + '=' * 50)

print('\n' + '=' * 50)

restantes = df.shape[0] - df['Type 2'].isna().sum()
print(f"Número de Pokémon sin tipo secundario: {restantes}")

print('\n' + '=' * 50)
```

```
==================================================
<StringArray>
[  'Poison',        nan,   'Flying',   'Dragon',   'Ground',    'Fairy',
    'Grass', 'Fighting',  'Psychic',    'Steel',      'Ice',     'Rock',
     'Dark',    'Water', 'Electric',     'Fire',    'Ghost',      'Bug',
   'Normal']
Length: 19, dtype: str

==================================================

==================================================
Número de Pokémon sin tipo secundario: 414

==================================================
```

Como vemos, el número de pokémon que nos quedan son justamente los que quedaban después de eliminar los nans.

Para poder trabajar más cómodamente con los datos, vamos a sustituir los 'nan' por 'None'.

In [18]:
```
df['Type 2'] = df['Type 2'].fillna('None')
print('\n' + '=' * 50)
print(df['Type 2'].unique())
print('\n' + '=' * 50)
```

```
==================================================
<StringArray>
[  'Poison',     'None',   'Flying',   'Dragon',   'Ground',    'Fairy',
    'Grass', 'Fighting',  'Psychic',    'Steel',      'Ice',     'Rock',
     'Dark',    'Water', 'Electric',     'Fire',    'Ghost',      'Bug',
   'Normal']
Length: 19, dtype: str

==================================================
```

Parte 1: SVR para evaluar las parejas de ataque y defensa físicos y especiales

---

Implementaremos un modelo SVR para detectar si hay cierta correlación entre las parejas de ataque-ataque especial y defensa-defensa especial. Escogeremos como variable X el ataque y evaluaremos la dependencia de Y, el ataque especial. Después, haremos lo mismo con las defensas.

Vamos a basarnos en el archivo de SVR que podemos encontrar en el aula virtual.

In [19]:
```
#Preparamos en primer lugar los datos

X = df['Attack'].values.reshape(-1, 1)
Y = df['Sp. Atk'].values.reshape(-1, 1)

#Escalamos los datos para que estén entre 0 y 1, ya que el SVR es sensible a la

from sklearn.preprocessing import MinMaxScaler
X_scaler = MinMaxScaler()
Y_scaler = MinMaxScaler()
X = X_scaler.fit_transform(X)
Y = Y_scaler.fit_transform(Y)
```

Continuaremos dividiendo los datos en conjuntos de entrenamiento, los cuales nos servirán para entrenar el modelo, y prueba, con los cuales se comprobará si este funciona correctamente.

In [20]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.33, random_state=100)
```

A continuación, tenemos que escoger los parámetros del modelo de regresión. Viendo los resultados que obtuvimos en clase y, como realmente no sabemos si hay una dependencia o no, vamos a escoger como kernel el 'rbf', que es el menos restrictivo y el

que puede caracterizar más casos. Además, escogeremos γ=1, que es valor por defecto, para ser más permisivos con el modelo. Por último, igual que en clase, escogeremos C=100

In [21]:
```
from sklearn.svm import SVR

svr_rbf = SVR(kernel='rbf', C=100, gamma=1, epsilon=.1)

# Creando el modelo y entrenando
reg = svr_rbf.fit(X_train,Y_train.reshape(-1))

# Prediciendo valores de entrenamiento
Y_train_hat = svr_rbf.predict(X_train)

# Prediciendo valores de validación
Y_test_hat = svr_rbf.predict(X_test)

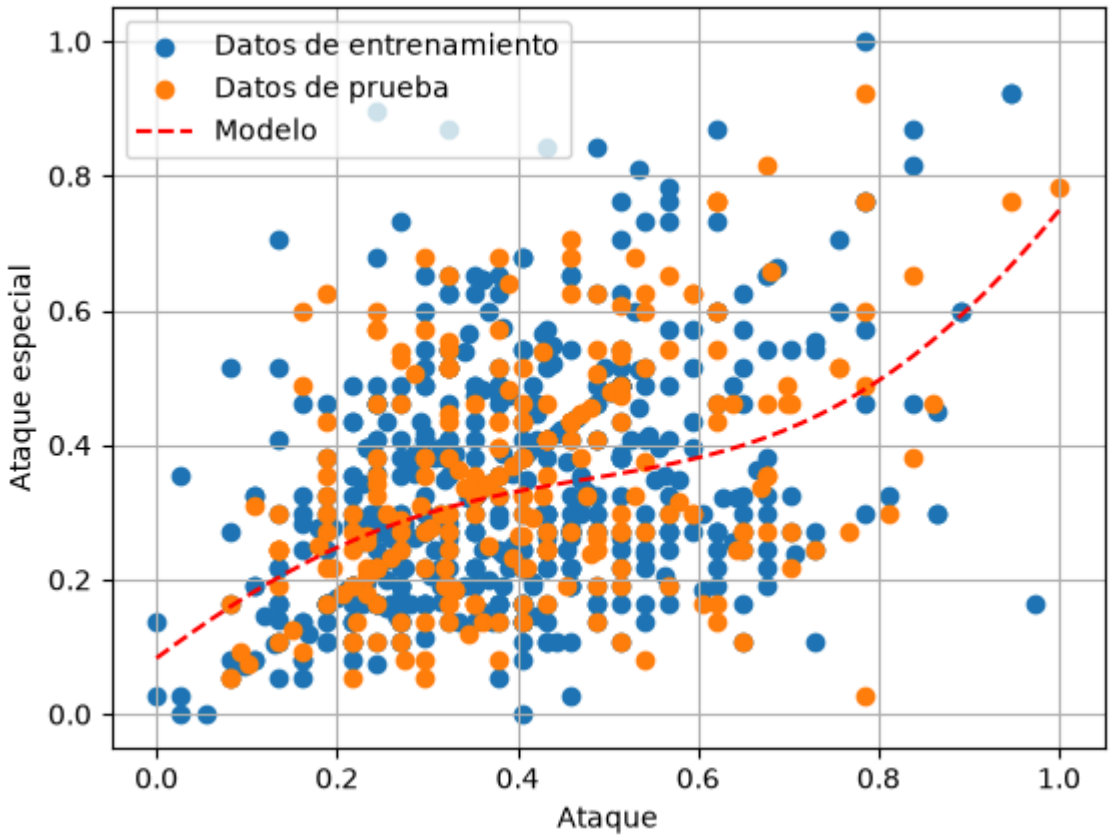


# Creamos un scatter plot con los datos de entrenamiento y validación
plt.scatter(X_train, Y_train)
plt.scatter(X_test, Y_test)

# En X_plot creamos un continuo de datos entre 0 y 1
X_plot = np.linspace(0,1).reshape(-1, 1)
Y_plot = svr_rbf.predict(X_plot)


# Graficamos el modelo
plt.plot(X_plot, Y_plot,"r--")
plt.xlabel('Ataque')
plt.ylabel('Ataque especial')
plt.legend(['Datos de entrenamiento', 'Datos de prueba', 'Modelo'])
plt.grid()
# Calculamos el error
from sklearn.metrics import r2_score
print("Entrenamiento", r2_score(Y_train, Y_train_hat))
print("Prueba", r2_score(Y_test, Y_test_hat))
```

```
Entrenamiento 0.15685371926245206
Prueba 0.1575360406902041
```



Podemos observar como el modelo no es nada bueno. La correlación es muy baja. Además, probando con el modelo lineal da un poco peor.

Por lo tanto, concluímos que, en general, el ataque especial no depende del ataque. ¿Qué pasa con la defensa?

In [22]:
```
#Hacemos todo exactamente igual pero con las variables de defensa y defensa espe

X = df['Defense'].values.reshape(-1, 1)
Y = df['Sp. Def'].values.reshape(-1, 1)


X_scaler = MinMaxScaler()
Y_scaler = MinMaxScaler()
X = X_scaler.fit_transform(X)
Y = Y_scaler.fit_transform(Y)

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.33, random_state=100)



svr_rbf = SVR(kernel='rbf', C=100, gamma=1, epsilon=.1)


reg = svr_rbf.fit(X_train,Y_train.reshape(-1))
Y_train_hat = svr_rbf.predict(X_train)
Y_test_hat = svr_rbf.predict(X_test)
plt.scatter(X_train, Y_train)
plt.scatter(X_test, Y_test)
```

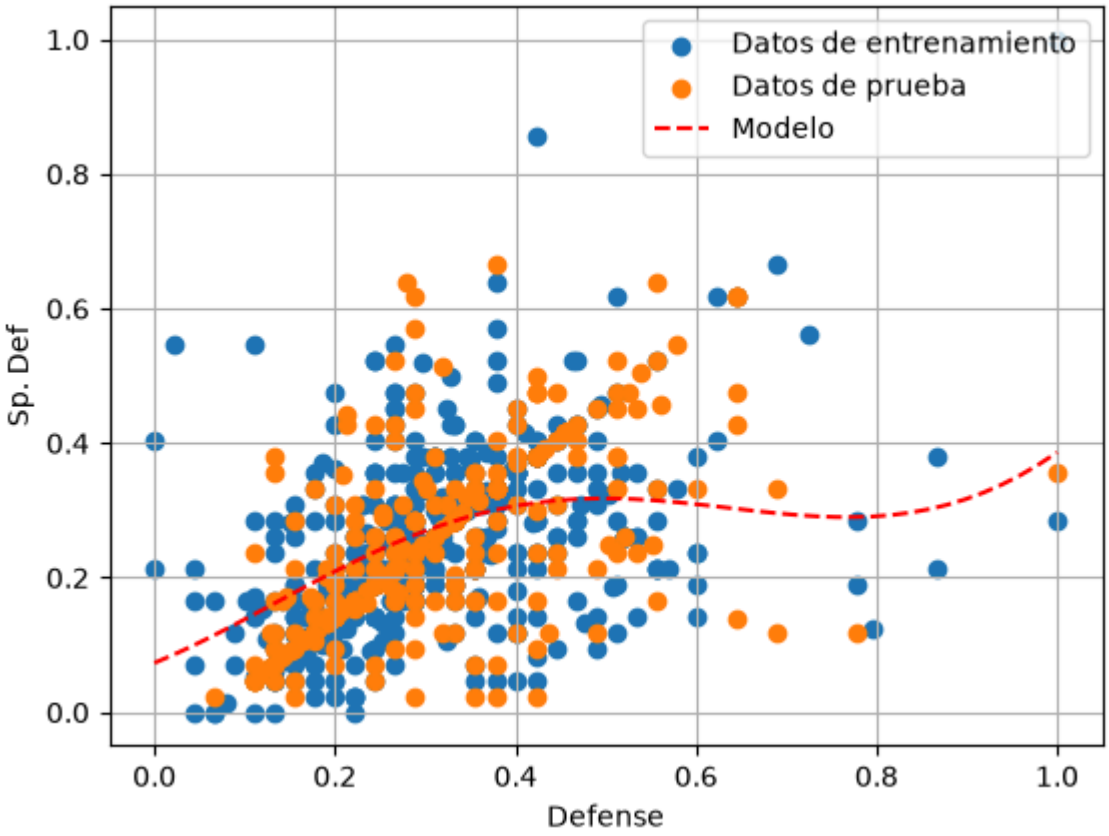
```
X_plot = np.linspace(0,1).reshape(-1, 1)
Y_plot = svr_rbf.predict(X_plot)



plt.plot(X_plot, Y_plot,"r--")
plt.xlabel('Defense'); plt.ylabel('Sp. Def')
plt.legend(['Datos de entrenamiento', 'Datos de prueba', 'Modelo'])
plt.grid()

print("Entrenamiento", r2_score(Y_train, Y_train_hat))
print("Prueba", r2_score(Y_test, Y_test_hat))
```

```
Entrenamiento 0.2591379987549308
Prueba 0.2788761849520067
```



Aunque con la defensa obtenemos un resultado un poco mejor, sigue sin ser gran cosa. Por tanto, podemos desmentir el hecho de que haya una relación entre estas estadísticas.

Ahora bien, ¿será entonces cierto que se distinguen el tipo lucha y psíquico?

Parte 2: ¿Psíquico o lucha?

Escogemos las variables que vamos a utilizar

Como comentamos anteriormente, vamos a intentar predecir el tipo de pokémon a partir de sus estadísiticas, restringiéndonos únicamente a los tipos psíquico y lucha. Para ello, nuestra variable X será todas las estadísticas del pokémon, mientras que la Y corresponderá a sus tipos (psíquico y lucha)

In [23]:
```
# Variables que utilizaremos
estadísticas = ['HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def', 'Speed']

# Seleccionamos únicamente Pokémon de tipo Psíquico y Lucha
df_filtrado = df[df['Type 1'].isin(['Psychic', 'Fighting']) | df['Type 2'].isin(


# Creamos la variable objetivo
# 1 = Psychic
# 0 = Fighting

df_filtrado['target'] = ((df_filtrado['Type 1'] == 'Psychic') | (df_filtrado['Ty

print('\n' + '=' * 50)

print('\nDistribución de las clases:')

print(df_filtrado[['Type 1','Type 2']].stack().value_counts())
print('\n' + '=' * 50)
print('\n' + '=' * 50)
print("\nNúmero de Pokémon seleccionados:", len(df_filtrado))
print('\n' + '=' * 50)
display(df_filtrado)
```

```
==================================================

Distribución de las clases:
Psychic     90
None        58
Fighting    53
Fire        10
Steel       10
Water        8
Flying       7
Grass        6
Fairy        6
Dark         6
Normal       4
Dragon       4
Rock         3
Ice          2
Bug          2
Ground       2
Poison       2
Ghost        1
Name: count, dtype: int64

==================================================

==================================================

Número de Pokémon seleccionados: 137

==================================================
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
61	56	Mankey	Fighting	None	305	40	80	35	35	45	70
62	57	Primeape	Fighting	None	455	65	105	60	60	70	95
67	62	Poliwrath	Water	Fighting	510	90	95	95	70	90	70
68	63	Abra	Psychic	None	310	25	20	15	105	55	90
69	64	Kadabra	Psychic	None	400	40	35	30	120	70	105
...	...	...	...	...	...	...	...	...	...	...	...
756	686	Inkay	Dark	Psychic	288	53	54	53	37	46	45
757	687	Malamar	Dark	Psychic	482	86	92	88	68	75	73
771	701	Hawlucha	Fighting	Flying	500	78	92	75	74	63	118
797	720	HoopaHoopa Confined	Psychic	Ghost	600	80	110	60	150	130	70
798	720	HoopaHoopa Unbound	Psychic	Dark	680	80	160	60	170	130	80

137 rows × 14 columns

En primer lugar, estudiaremos cómo se distribuyen las estadísticas dependiendo del tipo del pokémon para, de esta manera, ver si se aprecia alguna diferencia entre ellos.

In [24]:
```
# Distribución de Pokémon por tipo
plt.figure(figsize=(7, 5))

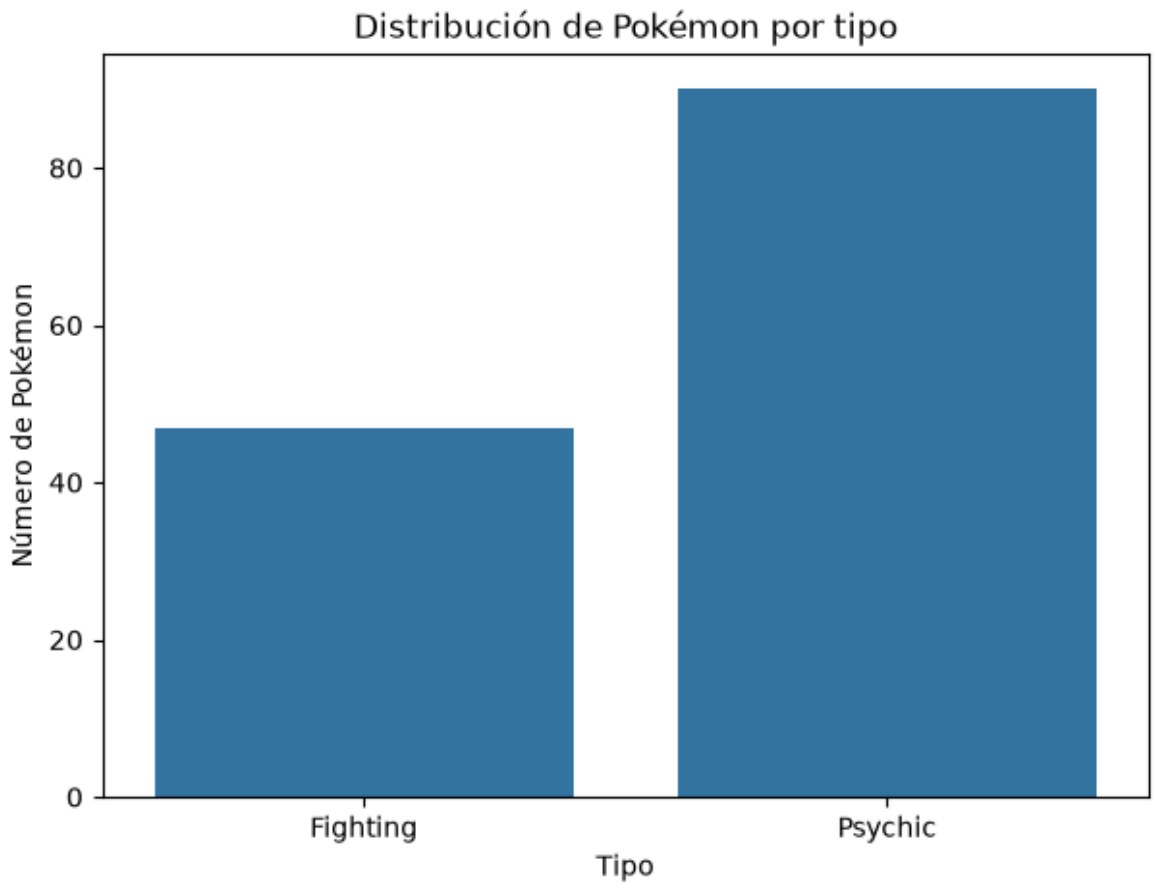
sns.countplot(data=df_filtrado, x='target')

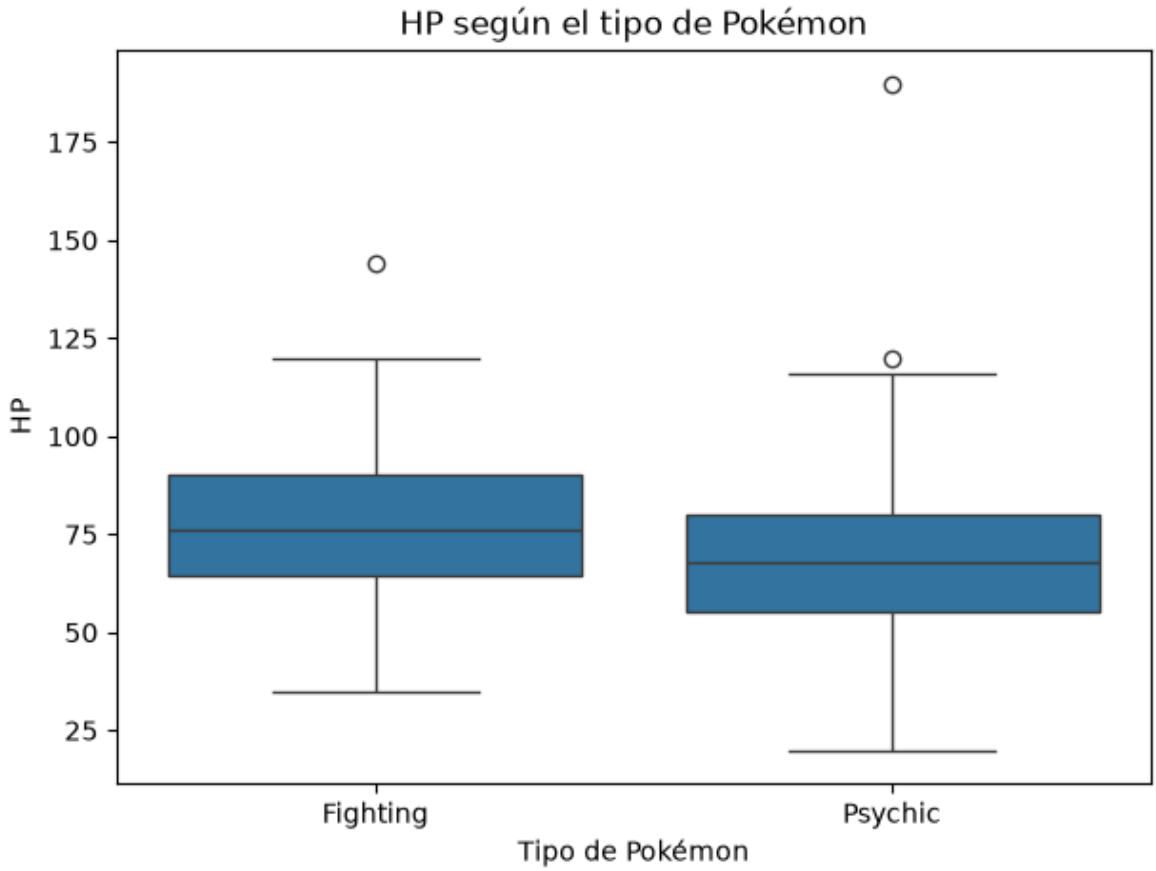
plt.title('Distribución de Pokémon por tipo')
plt.xticks([0, 1], ['Fighting', 'Psychic'])
plt.xlabel('Tipo')
plt.ylabel('Número de Pokémon')
plt.show()

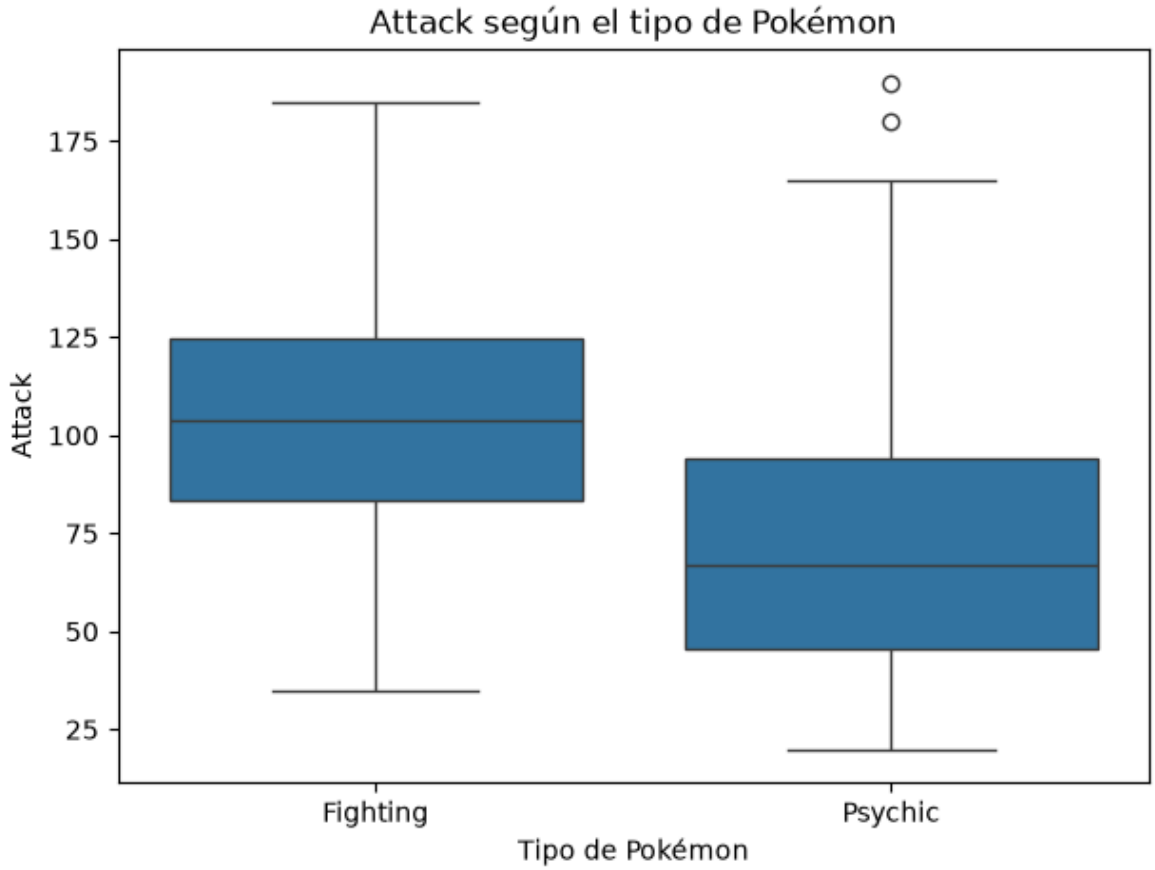

# Comparación de las estadísticas según el tipo
for variable in estadísticas:

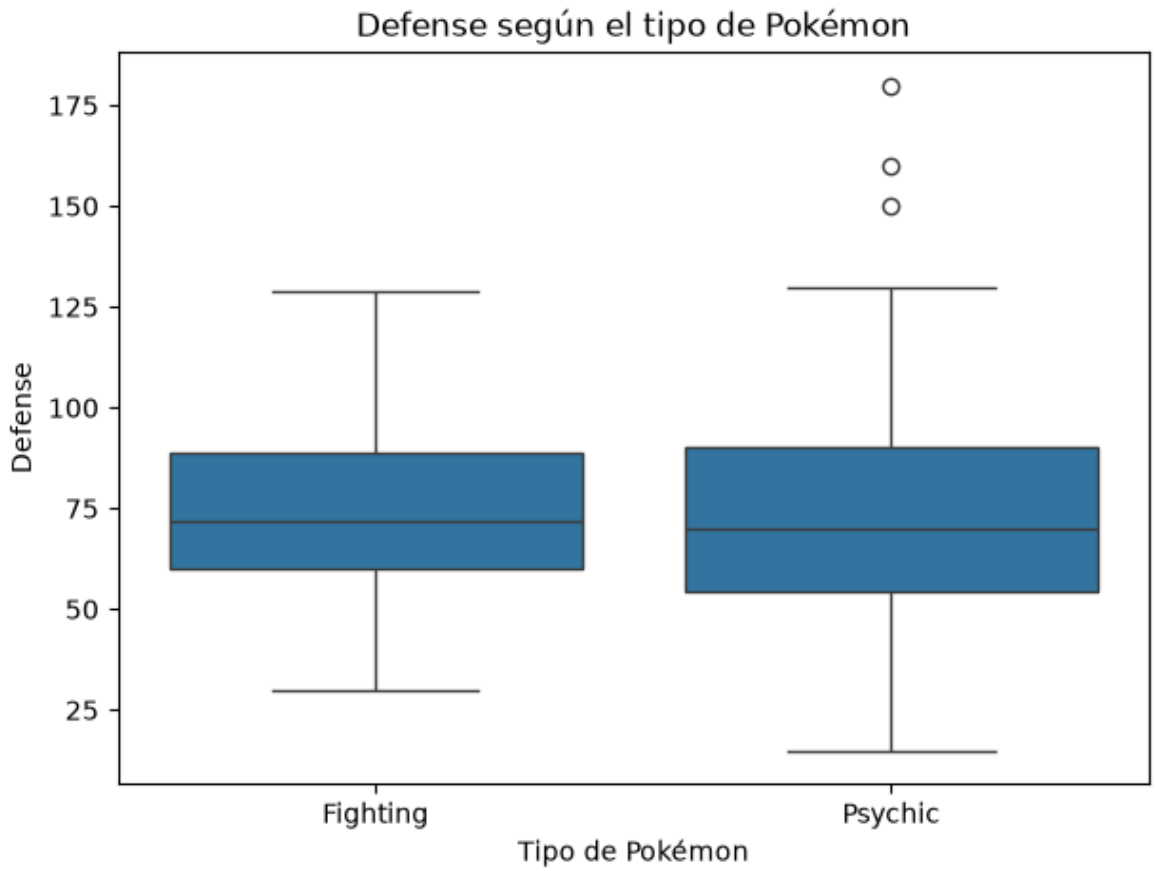
    plt.figure(figsize=(7, 5))

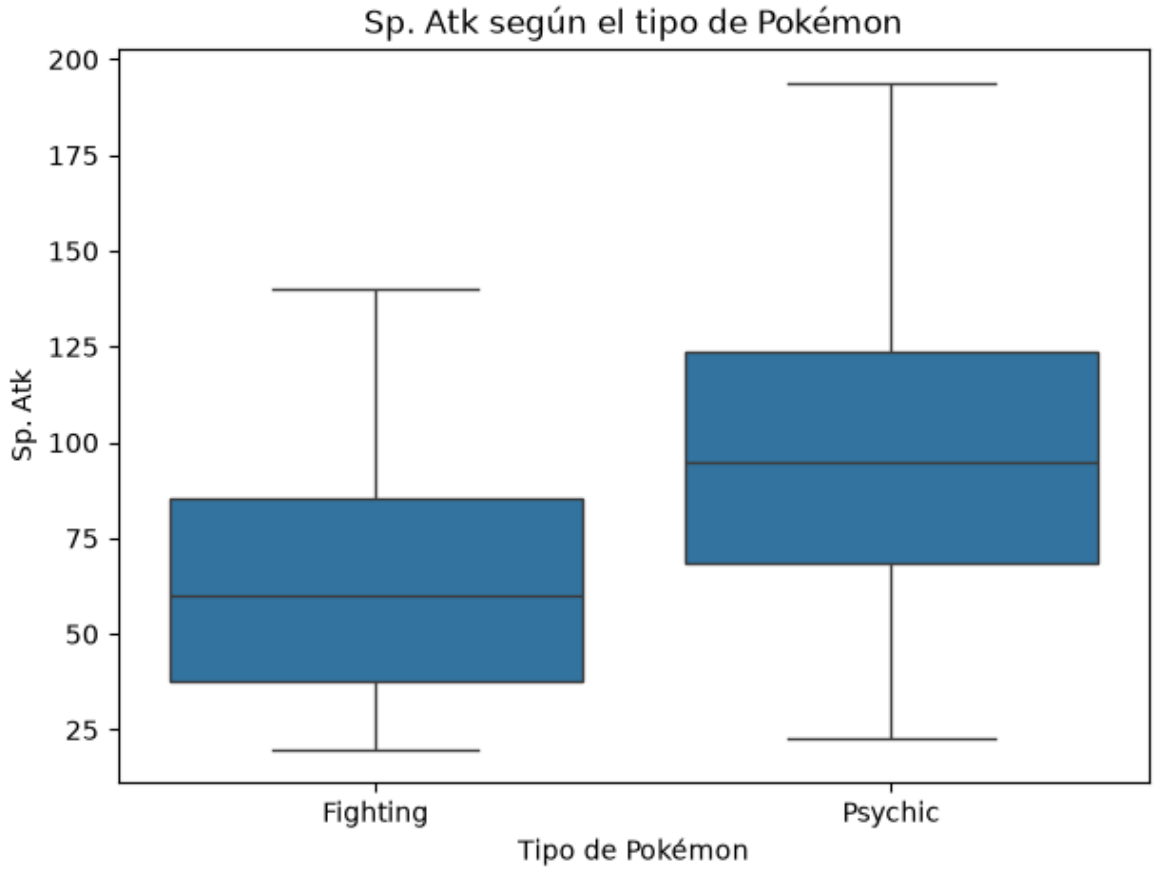
    sns.boxplot(data=df_filtrado, x='target', y=variable)
    plt.xticks([0, 1], ['Fighting', 'Psychic'])
    plt.xlabel('Tipo de Pokémon')
    plt.title(f'{variable} según el tipo de Pokémon')
    plt.show()
```

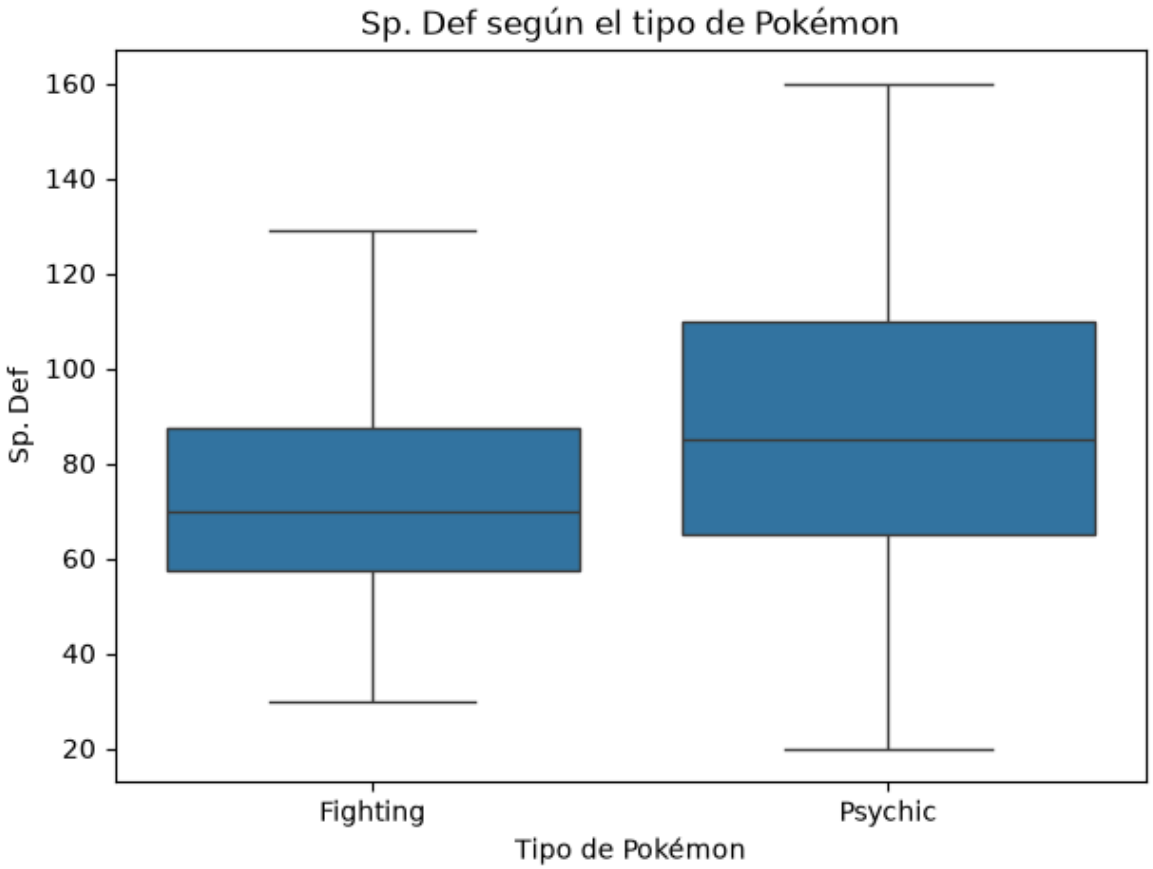


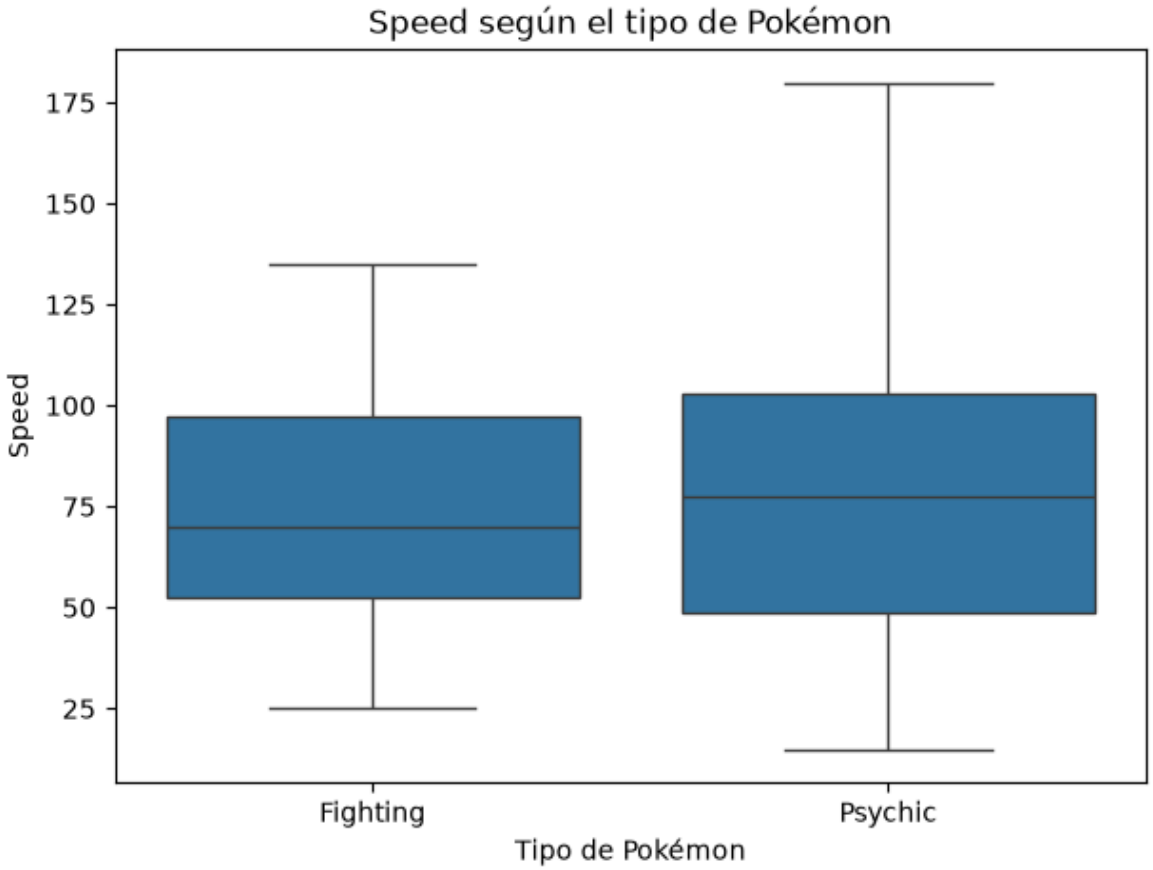

HP según el tipo de Pokémon
HP
175
150
125
100
75
50
25
Fighting
Psychic
Tipo de Pokémon


Attack según el tipo de Pokémon
Attack
175
150
125
100
75
50
25
Fighting
Psychic
Tipo de Pokémon


Defense según el tipo de Pokémon
Defense
175
150
125
100
75
50
25
Fighting
Psychic
Tipo de Pokémon


Sp. Atk según el tipo de Pokémon
Sp. Atk
200
175
150
125
100
75
50
25
Fighting
Psychic
Tipo de Pokémon





Empezamos a ver cosas interesantes. Atendiendo a las gráficas de ataque y ataque especial, vemos que sí hay una diferecia considerable.

A mayores podemos comprobar que tenemos bastantes más pokémon de tipo lucha que de tipo psíquico, además de que, como comentamos al principio, hay algunos que poseen ambos tipos a la vez.

Como hemos comentado anteriormente, para poder realizar el modelo de clasificación necesitamos dividir nuestros datos en varios conjuntos.

Primero, el conjunto de entrenamiento, que comprenderá los datos a partir de los cuales el modelo obtendrá las correlaciones que aplicará posteriormente. Este conjunto lo vamos a dividir en dos subconjuntos: uno para entrenar el algoritmo y otro para hacer una primera comprobación de su rendimiento.

Por último, vamos a escoger otro conjunto de prueba con el fin de evaluar cómo clasifica el modelo ante datos totalmente nuevos.

In [25]:
```
from sklearn.preprocessing import StandardScaler

# Variables predictoras
X = df_filtrado[estadísticas].values

# Variable objetivo
y = df_filtrado['target'].values

# Primera división: entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random

# División del entrenamiento para obtener validación
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.

# Escalado de las variables
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)


print('\n' + '=' * 50)


print('\nDatos de entrenamiento:', X_train_scaled.shape)
print('Datos de validación:', X_val_scaled.shape)
print('Datos de prueba:', X_test_scaled.shape)

print('\n' + '=' * 50)
```

```
==================================================

Datos de entrenamiento: (87, 6)
Datos de validación: (22, 6)
Datos de prueba: (28, 6)

==================================================
```

Hemos probado a variar los tamaños de los conjuntos de entrenamiento, validación y prueba para ver cómo esto afectaba al rendimiento del modelo. Intentamos escoger $\text{test_size} = 0.42$, igual que en algunos programas de clase. Sin embargo, obtuvimos resultados peores debido a que perder elementos en el conjunto de entrenamiento suponía perder precisión.

Construcción y entrenamiento de la red neuronal

Para construir nuestra red neuronal nos hemos inspirado en el modelo 'Detección de cáncer de mama' del aula virtual, debido a que nuestro tipo de clasificación es similar. De esta forma, creamos la siguiente arquitectura:

1. Una red de entrada de 20 neuronas tipo RELU
2. Capa de 20 neuronas tipo RELU
3. Capa de 20 neurones tipo RELU
4. Una última capa de una sola neurona tipo sigmoide

Escogeremos como función de pérdida '$\text{binary_crossentropy}$' la cual es particularmente útil para clasificaciones en dos categorías.

In [26]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.optimizers import SGD

from tensorflow.keras.callbacks import EarlyStopping

# Fijamos la semilla para reproducibilidad
np.random.seed(42)
tf.random.set_seed(42)

# Creación de la red neuronal
model = keras.Sequential([layers.Dense(20, activation='relu', input_shape=(X_tra

# Compilación del modelo
sgd = SGD(learning_rate=0.01)

model.compile(optimizer=sgd, loss='binary_crossentropy', metrics=['accuracy'])

# Resumen de la arquitectura
model.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
```

```
c:\Users\david\anaconda3\envs\tensorflow_env\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 20)	140
dense_1 (Dense)	(None, 20)	420
dense_2 (Dense)	(None, 20)	420
dense_3 (Dense)	(None, 1)	21

Total params: 1,001 (3.91 KB)

Trainable params: 1,001 (3.91 KB)

Non-trainable params: 0 (0.00 B)

In [27]:
```
# Early Stopping para evitar sobreajuste
early_stop = EarlyStopping(monitor='val_loss', patience=40, restore_best_weights

# Entrenamiento
print("Entrenando la red neuronal...")

history = model.fit(X_train_scaled, y_train, epochs=500, batch_size=10, validati

print("\n¡Entrenamiento completado!")

print("Número de épocas ejecutadas:", len(history.history['loss']))
```

```
Entrenando la red neuronal...
Epoch 1/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 2s 67ms/step - accuracy: 0.3793 - loss: 0.7426 - val_acc
uracy: 0.4545 - val_loss: 0.7246
Epoch 2/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.4598 - loss: 0.7126 - val_acc
uracy: 0.5000 - val_loss: 0.6948
Epoch 3/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5517 - loss: 0.6888 - val_acc
uracy: 0.5909 - val_loss: 0.6710
Epoch 4/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.5862 - loss: 0.6702 - val_acc
uracy: 0.6364 - val_loss: 0.6524
Epoch 5/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6322 - loss: 0.6551 - val_acc
uracy: 0.6818 - val_loss: 0.6369
Epoch 6/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.6552 - loss: 0.6424 - val_acc
uracy: 0.6818 - val_loss: 0.6238
Epoch 7/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6552 - loss: 0.6320 - val_acc
uracy: 0.6818 - val_loss: 0.6128
Epoch 8/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.6552 - loss: 0.6233 - val_acc
uracy: 0.6818 - val_loss: 0.6029
Epoch 9/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6552 - loss: 0.6158 - val_acc
uracy: 0.6818 - val_loss: 0.5940
Epoch 10/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6552 - loss: 0.6092 - val_acc
uracy: 0.6818 - val_loss: 0.5857
Epoch 11/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.6552 - loss: 0.6032 - val_acc
uracy: 0.6818 - val_loss: 0.5777
Epoch 12/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6552 - loss: 0.5978 - val_acc
uracy: 0.6818 - val_loss: 0.5701
Epoch 13/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6552 - loss: 0.5926 - val_acc
uracy: 0.6818 - val_loss: 0.5627
Epoch 14/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6552 - loss: 0.5877 - val_acc
uracy: 0.6818 - val_loss: 0.5553
Epoch 15/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.6552 - loss: 0.5830 - val_acc
uracy: 0.6818 - val_loss: 0.5481
Epoch 16/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.6552 - loss: 0.5783 - val_acc
uracy: 0.6818 - val_loss: 0.5408
Epoch 17/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.6552 - loss: 0.5737 - val_acc
uracy: 0.6818 - val_loss: 0.5336
Epoch 18/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.6552 - loss: 0.5691 - val_acc
uracy: 0.6818 - val_loss: 0.5263
Epoch 19/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.6552 - loss: 0.5646 - val_acc
uracy: 0.6818 - val_loss: 0.5190
Epoch 20/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.6667 - loss: 0.5601 - val_acc
```

```
uracy: 0.6818 - val_loss: 0.5116
Epoch 21/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.6667 - loss: 0.5557 - val_acc
uracy: 0.6818 - val_loss: 0.5040
Epoch 22/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6667 - loss: 0.5512 - val_acc
uracy: 0.6818 - val_loss: 0.4965
Epoch 23/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.6667 - loss: 0.5467 - val_acc
uracy: 0.6818 - val_loss: 0.4886
Epoch 24/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.6667 - loss: 0.5421 - val_acc
uracy: 0.6818 - val_loss: 0.4808
Epoch 25/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.6667 - loss: 0.5374 - val_acc
uracy: 0.6818 - val_loss: 0.4726
Epoch 26/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.6782 - loss: 0.5327 - val_acc
uracy: 0.6818 - val_loss: 0.4643
Epoch 27/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.6782 - loss: 0.5279 - val_acc
uracy: 0.6818 - val_loss: 0.4557
Epoch 28/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6782 - loss: 0.5230 - val_acc
uracy: 0.6818 - val_loss: 0.4469
Epoch 29/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.7011 - loss: 0.5182 - val_acc
uracy: 0.6818 - val_loss: 0.4383
Epoch 30/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.7126 - loss: 0.5134 - val_acc
uracy: 0.6818 - val_loss: 0.4297
Epoch 31/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7356 - loss: 0.5087 - val_acc
uracy: 0.6818 - val_loss: 0.4212
Epoch 32/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7471 - loss: 0.5039 - val_acc
uracy: 0.8182 - val_loss: 0.4126
Epoch 33/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.7701 - loss: 0.4992 - val_acc
uracy: 0.8182 - val_loss: 0.4039
Epoch 34/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.7701 - loss: 0.4945 - val_acc
uracy: 0.8182 - val_loss: 0.3949
Epoch 35/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7816 - loss: 0.4898 - val_acc
uracy: 0.8182 - val_loss: 0.3862
Epoch 36/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7816 - loss: 0.4851 - val_acc
uracy: 0.8636 - val_loss: 0.3774
Epoch 37/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.7816 - loss: 0.4805 - val_acc
uracy: 0.8636 - val_loss: 0.3686
Epoch 38/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8046 - loss: 0.4760 - val_acc
uracy: 0.8636 - val_loss: 0.3599
Epoch 39/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8276 - loss: 0.4715 - val_acc
uracy: 0.8636 - val_loss: 0.3514
Epoch 40/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8276 - loss: 0.4673 - val_acc
```

```
uracy: 0.9091 - val_loss: 0.3431
Epoch 41/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8276 - loss: 0.4630 - val_acc
uracy: 0.9545 - val_loss: 0.3348
Epoch 42/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8276 - loss: 0.4588 - val_acc
uracy: 0.9545 - val_loss: 0.3266
Epoch 43/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8276 - loss: 0.4546 - val_acc
uracy: 0.9545 - val_loss: 0.3186
Epoch 44/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8391 - loss: 0.4504 - val_acc
uracy: 0.9545 - val_loss: 0.3106
Epoch 45/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8391 - loss: 0.4464 - val_acc
uracy: 0.9545 - val_loss: 0.3029
Epoch 46/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.8391 - loss: 0.4424 - val_acc
uracy: 0.9545 - val_loss: 0.2954
Epoch 47/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8391 - loss: 0.4386 - val_acc
uracy: 0.9545 - val_loss: 0.2881
Epoch 48/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8391 - loss: 0.4350 - val_acc
uracy: 0.9545 - val_loss: 0.2810
Epoch 49/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8161 - loss: 0.4313 - val_acc
uracy: 0.9545 - val_loss: 0.2740
Epoch 50/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8161 - loss: 0.4278 - val_acc
uracy: 0.9545 - val_loss: 0.2672
Epoch 51/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8161 - loss: 0.4244 - val_acc
uracy: 0.9545 - val_loss: 0.2607
Epoch 52/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.8161 - loss: 0.4210 - val_acc
uracy: 0.9545 - val_loss: 0.2542
Epoch 53/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8161 - loss: 0.4178 - val_acc
uracy: 1.0000 - val_loss: 0.2480
Epoch 54/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8161 - loss: 0.4145 - val_acc
uracy: 1.0000 - val_loss: 0.2420
Epoch 55/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8161 - loss: 0.4115 - val_acc
uracy: 1.0000 - val_loss: 0.2362
Epoch 56/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8161 - loss: 0.4086 - val_acc
uracy: 1.0000 - val_loss: 0.2307
Epoch 57/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8161 - loss: 0.4057 - val_acc
uracy: 1.0000 - val_loss: 0.2254
Epoch 58/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8161 - loss: 0.4030 - val_acc
uracy: 1.0000 - val_loss: 0.2204
Epoch 59/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.8161 - loss: 0.4004 - val_acc
uracy: 1.0000 - val_loss: 0.2157
Epoch 60/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.8161 - loss: 0.3979 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.2111
Epoch 61/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.8161 - loss: 0.3955 - val_acc
uracy: 1.0000 - val_loss: 0.2068
Epoch 62/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.8161 - loss: 0.3932 - val_acc
uracy: 1.0000 - val_loss: 0.2025
Epoch 63/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.8276 - loss: 0.3909 - val_acc
uracy: 1.0000 - val_loss: 0.1983
Epoch 64/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.8276 - loss: 0.3887 - val_acc
uracy: 1.0000 - val_loss: 0.1943
Epoch 65/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8276 - loss: 0.3866 - val_acc
uracy: 1.0000 - val_loss: 0.1904
Epoch 66/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8276 - loss: 0.3845 - val_acc
uracy: 1.0000 - val_loss: 0.1867
Epoch 67/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.8276 - loss: 0.3825 - val_acc
uracy: 1.0000 - val_loss: 0.1830
Epoch 68/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8391 - loss: 0.3806 - val_acc
uracy: 1.0000 - val_loss: 0.1795
Epoch 69/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.8391 - loss: 0.3787 - val_acc
uracy: 1.0000 - val_loss: 0.1762
Epoch 70/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8391 - loss: 0.3768 - val_acc
uracy: 1.0000 - val_loss: 0.1729
Epoch 71/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8391 - loss: 0.3750 - val_acc
uracy: 1.0000 - val_loss: 0.1698
Epoch 72/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8391 - loss: 0.3733 - val_acc
uracy: 1.0000 - val_loss: 0.1668
Epoch 73/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8391 - loss: 0.3716 - val_acc
uracy: 1.0000 - val_loss: 0.1639
Epoch 74/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.8391 - loss: 0.3700 - val_acc
uracy: 1.0000 - val_loss: 0.1610
Epoch 75/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8391 - loss: 0.3684 - val_acc
uracy: 1.0000 - val_loss: 0.1583
Epoch 76/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8391 - loss: 0.3669 - val_acc
uracy: 1.0000 - val_loss: 0.1557
Epoch 77/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.8391 - loss: 0.3654 - val_acc
uracy: 1.0000 - val_loss: 0.1530
Epoch 78/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8391 - loss: 0.3639 - val_acc
uracy: 1.0000 - val_loss: 0.1505
Epoch 79/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8391 - loss: 0.3625 - val_acc
uracy: 1.0000 - val_loss: 0.1481
Epoch 80/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8391 - loss: 0.3610 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.1457
Epoch 81/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8391 - loss: 0.3596 - val_acc
uracy: 1.0000 - val_loss: 0.1434
Epoch 82/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8506 - loss: 0.3583 - val_acc
uracy: 1.0000 - val_loss: 0.1411
Epoch 83/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8506 - loss: 0.3569 - val_acc
uracy: 1.0000 - val_loss: 0.1390
Epoch 84/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8506 - loss: 0.3556 - val_acc
uracy: 1.0000 - val_loss: 0.1370
Epoch 85/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8506 - loss: 0.3544 - val_acc
uracy: 1.0000 - val_loss: 0.1350
Epoch 86/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8506 - loss: 0.3531 - val_acc
uracy: 1.0000 - val_loss: 0.1332
Epoch 87/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.8506 - loss: 0.3519 - val_acc
uracy: 1.0000 - val_loss: 0.1314
Epoch 88/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8506 - loss: 0.3506 - val_acc
uracy: 1.0000 - val_loss: 0.1296
Epoch 89/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8506 - loss: 0.3495 - val_acc
uracy: 1.0000 - val_loss: 0.1278
Epoch 90/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8506 - loss: 0.3483 - val_acc
uracy: 1.0000 - val_loss: 0.1261
Epoch 91/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8506 - loss: 0.3472 - val_acc
uracy: 1.0000 - val_loss: 0.1245
Epoch 92/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8506 - loss: 0.3460 - val_acc
uracy: 1.0000 - val_loss: 0.1230
Epoch 93/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8506 - loss: 0.3448 - val_acc
uracy: 1.0000 - val_loss: 0.1215
Epoch 94/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8506 - loss: 0.3436 - val_acc
uracy: 1.0000 - val_loss: 0.1200
Epoch 95/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.8506 - loss: 0.3424 - val_acc
uracy: 1.0000 - val_loss: 0.1185
Epoch 96/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8506 - loss: 0.3413 - val_acc
uracy: 1.0000 - val_loss: 0.1171
Epoch 97/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8621 - loss: 0.3402 - val_acc
uracy: 1.0000 - val_loss: 0.1157
Epoch 98/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8621 - loss: 0.3392 - val_acc
uracy: 1.0000 - val_loss: 0.1145
Epoch 99/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8621 - loss: 0.3382 - val_acc
uracy: 1.0000 - val_loss: 0.1132
Epoch 100/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.3371 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.1120
Epoch 101/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8621 - loss: 0.3360 - val_acc
uracy: 1.0000 - val_loss: 0.1109
Epoch 102/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8621 - loss: 0.3351 - val_acc
uracy: 1.0000 - val_loss: 0.1098
Epoch 103/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3340 - val_acc
uracy: 1.0000 - val_loss: 0.1087
Epoch 104/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3330 - val_acc
uracy: 1.0000 - val_loss: 0.1076
Epoch 105/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3321 - val_acc
uracy: 1.0000 - val_loss: 0.1065
Epoch 106/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3311 - val_acc
uracy: 1.0000 - val_loss: 0.1055
Epoch 107/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3301 - val_acc
uracy: 1.0000 - val_loss: 0.1045
Epoch 108/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3292 - val_acc
uracy: 1.0000 - val_loss: 0.1035
Epoch 109/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3282 - val_acc
uracy: 1.0000 - val_loss: 0.1026
Epoch 110/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.3273 - val_acc
uracy: 1.0000 - val_loss: 0.1017
Epoch 111/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3265 - val_acc
uracy: 1.0000 - val_loss: 0.1008
Epoch 112/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8621 - loss: 0.3255 - val_acc
uracy: 1.0000 - val_loss: 0.0999
Epoch 113/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.8621 - loss: 0.3247 - val_acc
uracy: 1.0000 - val_loss: 0.0991
Epoch 114/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8621 - loss: 0.3238 - val_acc
uracy: 1.0000 - val_loss: 0.0982
Epoch 115/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3229 - val_acc
uracy: 1.0000 - val_loss: 0.0973
Epoch 116/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3220 - val_acc
uracy: 1.0000 - val_loss: 0.0966
Epoch 117/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3211 - val_acc
uracy: 1.0000 - val_loss: 0.0958
Epoch 118/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3201 - val_acc
uracy: 1.0000 - val_loss: 0.0949
Epoch 119/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3192 - val_acc
uracy: 1.0000 - val_loss: 0.0942
Epoch 120/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.3183 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0935
Epoch 121/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8621 - loss: 0.3173 - val_acc
uracy: 1.0000 - val_loss: 0.0927
Epoch 122/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3164 - val_acc
uracy: 1.0000 - val_loss: 0.0921
Epoch 123/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.3154 - val_acc
uracy: 1.0000 - val_loss: 0.0914
Epoch 124/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3145 - val_acc
uracy: 1.0000 - val_loss: 0.0907
Epoch 125/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3136 - val_acc
uracy: 1.0000 - val_loss: 0.0901
Epoch 126/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8621 - loss: 0.3126 - val_acc
uracy: 1.0000 - val_loss: 0.0895
Epoch 127/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3116 - val_acc
uracy: 1.0000 - val_loss: 0.0888
Epoch 128/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.3106 - val_acc
uracy: 1.0000 - val_loss: 0.0883
Epoch 129/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3096 - val_acc
uracy: 1.0000 - val_loss: 0.0877
Epoch 130/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3086 - val_acc
uracy: 1.0000 - val_loss: 0.0871
Epoch 131/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.8621 - loss: 0.3077 - val_acc
uracy: 1.0000 - val_loss: 0.0865
Epoch 132/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.3067 - val_acc
uracy: 1.0000 - val_loss: 0.0858
Epoch 133/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.3058 - val_acc
uracy: 1.0000 - val_loss: 0.0853
Epoch 134/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8621 - loss: 0.3048 - val_acc
uracy: 1.0000 - val_loss: 0.0847
Epoch 135/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8621 - loss: 0.3038 - val_acc
uracy: 1.0000 - val_loss: 0.0841
Epoch 136/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8621 - loss: 0.3029 - val_acc
uracy: 1.0000 - val_loss: 0.0836
Epoch 137/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.3017 - val_acc
uracy: 1.0000 - val_loss: 0.0830
Epoch 138/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8621 - loss: 0.3008 - val_acc
uracy: 1.0000 - val_loss: 0.0825
Epoch 139/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8621 - loss: 0.2997 - val_acc
uracy: 1.0000 - val_loss: 0.0819
Epoch 140/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.2987 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0814
Epoch 141/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.8621 - loss: 0.2977 - val_acc
uracy: 1.0000 - val_loss: 0.0808
Epoch 142/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.2967 - val_acc
uracy: 1.0000 - val_loss: 0.0802
Epoch 143/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8621 - loss: 0.2958 - val_acc
uracy: 1.0000 - val_loss: 0.0798
Epoch 144/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.2947 - val_acc
uracy: 1.0000 - val_loss: 0.0793
Epoch 145/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8621 - loss: 0.2937 - val_acc
uracy: 1.0000 - val_loss: 0.0787
Epoch 146/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8621 - loss: 0.2926 - val_acc
uracy: 1.0000 - val_loss: 0.0783
Epoch 147/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.2916 - val_acc
uracy: 1.0000 - val_loss: 0.0778
Epoch 148/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.8621 - loss: 0.2905 - val_acc
uracy: 1.0000 - val_loss: 0.0774
Epoch 149/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8621 - loss: 0.2894 - val_acc
uracy: 1.0000 - val_loss: 0.0769
Epoch 150/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8736 - loss: 0.2887 - val_acc
uracy: 1.0000 - val_loss: 0.0764
Epoch 151/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8736 - loss: 0.2876 - val_acc
uracy: 1.0000 - val_loss: 0.0760
Epoch 152/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8736 - loss: 0.2866 - val_acc
uracy: 1.0000 - val_loss: 0.0755
Epoch 153/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8851 - loss: 0.2856 - val_acc
uracy: 1.0000 - val_loss: 0.0750
Epoch 154/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8851 - loss: 0.2847 - val_acc
uracy: 1.0000 - val_loss: 0.0746
Epoch 155/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2839 - val_acc
uracy: 1.0000 - val_loss: 0.0742
Epoch 156/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8851 - loss: 0.2828 - val_acc
uracy: 1.0000 - val_loss: 0.0739
Epoch 157/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8851 - loss: 0.2820 - val_acc
uracy: 1.0000 - val_loss: 0.0734
Epoch 158/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8851 - loss: 0.2810 - val_acc
uracy: 1.0000 - val_loss: 0.0730
Epoch 159/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2802 - val_acc
uracy: 1.0000 - val_loss: 0.0726
Epoch 160/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2793 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0721
Epoch 161/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2784 - val_acc
uracy: 1.0000 - val_loss: 0.0718
Epoch 162/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8851 - loss: 0.2775 - val_acc
uracy: 1.0000 - val_loss: 0.0714
Epoch 163/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.8851 - loss: 0.2765 - val_acc
uracy: 1.0000 - val_loss: 0.0710
Epoch 164/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2756 - val_acc
uracy: 1.0000 - val_loss: 0.0706
Epoch 165/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.8851 - loss: 0.2748 - val_acc
uracy: 1.0000 - val_loss: 0.0703
Epoch 166/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2737 - val_acc
uracy: 1.0000 - val_loss: 0.0698
Epoch 167/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2729 - val_acc
uracy: 1.0000 - val_loss: 0.0695
Epoch 168/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2718 - val_acc
uracy: 1.0000 - val_loss: 0.0691
Epoch 169/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8851 - loss: 0.2709 - val_acc
uracy: 1.0000 - val_loss: 0.0688
Epoch 170/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2700 - val_acc
uracy: 1.0000 - val_loss: 0.0684
Epoch 171/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8851 - loss: 0.2689 - val_acc
uracy: 1.0000 - val_loss: 0.0680
Epoch 172/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8851 - loss: 0.2681 - val_acc
uracy: 1.0000 - val_loss: 0.0676
Epoch 173/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2672 - val_acc
uracy: 1.0000 - val_loss: 0.0673
Epoch 174/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8851 - loss: 0.2663 - val_acc
uracy: 1.0000 - val_loss: 0.0669
Epoch 175/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8966 - loss: 0.2653 - val_acc
uracy: 1.0000 - val_loss: 0.0665
Epoch 176/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8966 - loss: 0.2646 - val_acc
uracy: 1.0000 - val_loss: 0.0662
Epoch 177/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8966 - loss: 0.2637 - val_acc
uracy: 1.0000 - val_loss: 0.0658
Epoch 178/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8966 - loss: 0.2627 - val_acc
uracy: 1.0000 - val_loss: 0.0655
Epoch 179/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8966 - loss: 0.2619 - val_acc
uracy: 1.0000 - val_loss: 0.0652
Epoch 180/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8966 - loss: 0.2611 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0649
Epoch 181/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8966 - loss: 0.2602 - val_acc
uracy: 1.0000 - val_loss: 0.0646
Epoch 182/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8966 - loss: 0.2593 - val_acc
uracy: 1.0000 - val_loss: 0.0642
Epoch 183/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8966 - loss: 0.2584 - val_acc
uracy: 1.0000 - val_loss: 0.0638
Epoch 184/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8966 - loss: 0.2576 - val_acc
uracy: 1.0000 - val_loss: 0.0635
Epoch 185/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9080 - loss: 0.2567 - val_acc
uracy: 1.0000 - val_loss: 0.0631
Epoch 186/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8966 - loss: 0.2560 - val_acc
uracy: 1.0000 - val_loss: 0.0629
Epoch 187/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9080 - loss: 0.2550 - val_acc
uracy: 1.0000 - val_loss: 0.0625
Epoch 188/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9080 - loss: 0.2543 - val_acc
uracy: 1.0000 - val_loss: 0.0621
Epoch 189/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9080 - loss: 0.2534 - val_acc
uracy: 1.0000 - val_loss: 0.0619
Epoch 190/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9080 - loss: 0.2526 - val_acc
uracy: 1.0000 - val_loss: 0.0616
Epoch 191/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9080 - loss: 0.2519 - val_acc
uracy: 1.0000 - val_loss: 0.0614
Epoch 192/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9080 - loss: 0.2508 - val_acc
uracy: 1.0000 - val_loss: 0.0611
Epoch 193/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9080 - loss: 0.2502 - val_acc
uracy: 1.0000 - val_loss: 0.0609
Epoch 194/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9080 - loss: 0.2492 - val_acc
uracy: 1.0000 - val_loss: 0.0607
Epoch 195/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9080 - loss: 0.2485 - val_acc
uracy: 1.0000 - val_loss: 0.0604
Epoch 196/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9080 - loss: 0.2477 - val_acc
uracy: 1.0000 - val_loss: 0.0602
Epoch 197/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9080 - loss: 0.2472 - val_acc
uracy: 1.0000 - val_loss: 0.0600
Epoch 198/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9080 - loss: 0.2460 - val_acc
uracy: 1.0000 - val_loss: 0.0597
Epoch 199/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9080 - loss: 0.2455 - val_acc
uracy: 1.0000 - val_loss: 0.0596
Epoch 200/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9080 - loss: 0.2446 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0593
Epoch 201/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9080 - loss: 0.2440 - val_acc
uracy: 1.0000 - val_loss: 0.0591
Epoch 202/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9080 - loss: 0.2431 - val_acc
uracy: 1.0000 - val_loss: 0.0588
Epoch 203/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9080 - loss: 0.2423 - val_acc
uracy: 1.0000 - val_loss: 0.0586
Epoch 204/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9080 - loss: 0.2416 - val_acc
uracy: 1.0000 - val_loss: 0.0584
Epoch 205/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.9080 - loss: 0.2408 - val_acc
uracy: 1.0000 - val_loss: 0.0583
Epoch 206/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9080 - loss: 0.2401 - val_acc
uracy: 1.0000 - val_loss: 0.0580
Epoch 207/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - accuracy: 0.9080 - loss: 0.2394 - val_acc
uracy: 1.0000 - val_loss: 0.0579
Epoch 208/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9080 - loss: 0.2386 - val_acc
uracy: 1.0000 - val_loss: 0.0577
Epoch 209/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9080 - loss: 0.2380 - val_acc
uracy: 1.0000 - val_loss: 0.0575
Epoch 210/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9080 - loss: 0.2371 - val_acc
uracy: 1.0000 - val_loss: 0.0573
Epoch 211/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9080 - loss: 0.2365 - val_acc
uracy: 1.0000 - val_loss: 0.0570
Epoch 212/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9080 - loss: 0.2358 - val_acc
uracy: 1.0000 - val_loss: 0.0569
Epoch 213/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9195 - loss: 0.2351 - val_acc
uracy: 1.0000 - val_loss: 0.0568
Epoch 214/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9310 - loss: 0.2344 - val_acc
uracy: 1.0000 - val_loss: 0.0566
Epoch 215/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9310 - loss: 0.2337 - val_acc
uracy: 1.0000 - val_loss: 0.0565
Epoch 216/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.9310 - loss: 0.2329 - val_acc
uracy: 1.0000 - val_loss: 0.0563
Epoch 217/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9310 - loss: 0.2323 - val_acc
uracy: 1.0000 - val_loss: 0.0561
Epoch 218/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.9310 - loss: 0.2315 - val_acc
uracy: 1.0000 - val_loss: 0.0559
Epoch 219/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.2308 - val_acc
uracy: 1.0000 - val_loss: 0.0558
Epoch 220/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9310 - loss: 0.2301 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0557
Epoch 221/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9310 - loss: 0.2294 - val_acc
uracy: 1.0000 - val_loss: 0.0555
Epoch 222/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9310 - loss: 0.2286 - val_acc
uracy: 1.0000 - val_loss: 0.0554
Epoch 223/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9310 - loss: 0.2281 - val_acc
uracy: 1.0000 - val_loss: 0.0552
Epoch 224/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9310 - loss: 0.2273 - val_acc
uracy: 1.0000 - val_loss: 0.0551
Epoch 225/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9310 - loss: 0.2266 - val_acc
uracy: 1.0000 - val_loss: 0.0549
Epoch 226/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9310 - loss: 0.2258 - val_acc
uracy: 1.0000 - val_loss: 0.0548
Epoch 227/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.2253 - val_acc
uracy: 1.0000 - val_loss: 0.0547
Epoch 228/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9310 - loss: 0.2244 - val_acc
uracy: 1.0000 - val_loss: 0.0546
Epoch 229/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9310 - loss: 0.2237 - val_acc
uracy: 1.0000 - val_loss: 0.0543
Epoch 230/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9310 - loss: 0.2232 - val_acc
uracy: 1.0000 - val_loss: 0.0542
Epoch 231/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9310 - loss: 0.2225 - val_acc
uracy: 1.0000 - val_loss: 0.0540
Epoch 232/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9310 - loss: 0.2217 - val_acc
uracy: 1.0000 - val_loss: 0.0539
Epoch 233/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.9310 - loss: 0.2212 - val_acc
uracy: 1.0000 - val_loss: 0.0537
Epoch 234/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9310 - loss: 0.2204 - val_acc
uracy: 1.0000 - val_loss: 0.0535
Epoch 235/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.9310 - loss: 0.2198 - val_acc
uracy: 1.0000 - val_loss: 0.0534
Epoch 236/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - accuracy: 0.9310 - loss: 0.2190 - val_acc
uracy: 1.0000 - val_loss: 0.0533
Epoch 237/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9310 - loss: 0.2185 - val_acc
uracy: 1.0000 - val_loss: 0.0530
Epoch 238/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9310 - loss: 0.2178 - val_acc
uracy: 1.0000 - val_loss: 0.0528
Epoch 239/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.2173 - val_acc
uracy: 1.0000 - val_loss: 0.0529
Epoch 240/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9310 - loss: 0.2166 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0527
Epoch 241/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9310 - loss: 0.2161 - val_acc
uracy: 1.0000 - val_loss: 0.0527
Epoch 242/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9310 - loss: 0.2153 - val_acc
uracy: 1.0000 - val_loss: 0.0526
Epoch 243/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9310 - loss: 0.2148 - val_acc
uracy: 1.0000 - val_loss: 0.0525
Epoch 244/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9310 - loss: 0.2141 - val_acc
uracy: 1.0000 - val_loss: 0.0523
Epoch 245/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9310 - loss: 0.2136 - val_acc
uracy: 1.0000 - val_loss: 0.0522
Epoch 246/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9310 - loss: 0.2131 - val_acc
uracy: 1.0000 - val_loss: 0.0520
Epoch 247/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9310 - loss: 0.2125 - val_acc
uracy: 1.0000 - val_loss: 0.0520
Epoch 248/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9310 - loss: 0.2118 - val_acc
uracy: 1.0000 - val_loss: 0.0519
Epoch 249/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9310 - loss: 0.2113 - val_acc
uracy: 1.0000 - val_loss: 0.0518
Epoch 250/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9310 - loss: 0.2108 - val_acc
uracy: 1.0000 - val_loss: 0.0516
Epoch 251/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.9310 - loss: 0.2101 - val_acc
uracy: 1.0000 - val_loss: 0.0515
Epoch 252/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.2095 - val_acc
uracy: 1.0000 - val_loss: 0.0512
Epoch 253/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.2092 - val_acc
uracy: 1.0000 - val_loss: 0.0512
Epoch 254/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9310 - loss: 0.2085 - val_acc
uracy: 1.0000 - val_loss: 0.0511
Epoch 255/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.2078 - val_acc
uracy: 1.0000 - val_loss: 0.0510
Epoch 256/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.2073 - val_acc
uracy: 1.0000 - val_loss: 0.0508
Epoch 257/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9310 - loss: 0.2068 - val_acc
uracy: 1.0000 - val_loss: 0.0506
Epoch 258/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.2063 - val_acc
uracy: 1.0000 - val_loss: 0.0502
Epoch 259/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9310 - loss: 0.2057 - val_acc
uracy: 1.0000 - val_loss: 0.0503
Epoch 260/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.2052 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0500
Epoch 261/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.2045 - val_acc
uracy: 1.0000 - val_loss: 0.0497
Epoch 262/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.2041 - val_acc
uracy: 1.0000 - val_loss: 0.0496
Epoch 263/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.2037 - val_acc
uracy: 1.0000 - val_loss: 0.0492
Epoch 264/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.2030 - val_acc
uracy: 1.0000 - val_loss: 0.0491
Epoch 265/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.2026 - val_acc
uracy: 1.0000 - val_loss: 0.0489
Epoch 266/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.9310 - loss: 0.2020 - val_acc
uracy: 1.0000 - val_loss: 0.0485
Epoch 267/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.2016 - val_acc
uracy: 1.0000 - val_loss: 0.0484
Epoch 268/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.9310 - loss: 0.2011 - val_acc
uracy: 1.0000 - val_loss: 0.0482
Epoch 269/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.9310 - loss: 0.2005 - val_acc
uracy: 1.0000 - val_loss: 0.0482
Epoch 270/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1999 - val_acc
uracy: 1.0000 - val_loss: 0.0477
Epoch 271/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1996 - val_acc
uracy: 1.0000 - val_loss: 0.0476
Epoch 272/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1991 - val_acc
uracy: 1.0000 - val_loss: 0.0475
Epoch 273/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1985 - val_acc
uracy: 1.0000 - val_loss: 0.0471
Epoch 274/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9310 - loss: 0.1980 - val_acc
uracy: 1.0000 - val_loss: 0.0470
Epoch 275/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.1974 - val_acc
uracy: 1.0000 - val_loss: 0.0469
Epoch 276/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9310 - loss: 0.1970 - val_acc
uracy: 1.0000 - val_loss: 0.0464
Epoch 277/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1966 - val_acc
uracy: 1.0000 - val_loss: 0.0463
Epoch 278/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1960 - val_acc
uracy: 1.0000 - val_loss: 0.0463
Epoch 279/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1955 - val_acc
uracy: 1.0000 - val_loss: 0.0458
Epoch 280/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1949 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0458
Epoch 281/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1947 - val_acc
uracy: 1.0000 - val_loss: 0.0456
Epoch 282/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9310 - loss: 0.1942 - val_acc
uracy: 1.0000 - val_loss: 0.0455
Epoch 283/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1935 - val_acc
uracy: 1.0000 - val_loss: 0.0453
Epoch 284/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1932 - val_acc
uracy: 1.0000 - val_loss: 0.0451
Epoch 285/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9310 - loss: 0.1927 - val_acc
uracy: 1.0000 - val_loss: 0.0450
Epoch 286/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1923 - val_acc
uracy: 1.0000 - val_loss: 0.0448
Epoch 287/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1916 - val_acc
uracy: 1.0000 - val_loss: 0.0447
Epoch 288/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1912 - val_acc
uracy: 1.0000 - val_loss: 0.0446
Epoch 289/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1906 - val_acc
uracy: 1.0000 - val_loss: 0.0444
Epoch 290/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1903 - val_acc
uracy: 1.0000 - val_loss: 0.0442
Epoch 291/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1898 - val_acc
uracy: 1.0000 - val_loss: 0.0440
Epoch 292/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1892 - val_acc
uracy: 1.0000 - val_loss: 0.0439
Epoch 293/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1890 - val_acc
uracy: 1.0000 - val_loss: 0.0437
Epoch 294/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1883 - val_acc
uracy: 1.0000 - val_loss: 0.0437
Epoch 295/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1880 - val_acc
uracy: 1.0000 - val_loss: 0.0434
Epoch 296/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9310 - loss: 0.1875 - val_acc
uracy: 1.0000 - val_loss: 0.0434
Epoch 297/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9310 - loss: 0.1868 - val_acc
uracy: 1.0000 - val_loss: 0.0433
Epoch 298/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1866 - val_acc
uracy: 1.0000 - val_loss: 0.0432
Epoch 299/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1859 - val_acc
uracy: 1.0000 - val_loss: 0.0430
Epoch 300/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1857 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0430
Epoch 301/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1852 - val_acc
uracy: 1.0000 - val_loss: 0.0428
Epoch 302/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1848 - val_acc
uracy: 1.0000 - val_loss: 0.0427
Epoch 303/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1842 - val_acc
uracy: 1.0000 - val_loss: 0.0427
Epoch 304/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1836 - val_acc
uracy: 1.0000 - val_loss: 0.0426
Epoch 305/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1834 - val_acc
uracy: 1.0000 - val_loss: 0.0425
Epoch 306/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9195 - loss: 0.1827 - val_acc
uracy: 1.0000 - val_loss: 0.0424
Epoch 307/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9195 - loss: 0.1826 - val_acc
uracy: 1.0000 - val_loss: 0.0422
Epoch 308/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9425 - loss: 0.1818 - val_acc
uracy: 1.0000 - val_loss: 0.0422
Epoch 309/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1816 - val_acc
uracy: 1.0000 - val_loss: 0.0423
Epoch 310/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9425 - loss: 0.1810 - val_acc
uracy: 1.0000 - val_loss: 0.0423
Epoch 311/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9310 - loss: 0.1806 - val_acc
uracy: 1.0000 - val_loss: 0.0420
Epoch 312/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1798 - val_acc
uracy: 1.0000 - val_loss: 0.0421
Epoch 313/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1792 - val_acc
uracy: 1.0000 - val_loss: 0.0419
Epoch 314/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9425 - loss: 0.1791 - val_acc
uracy: 1.0000 - val_loss: 0.0419
Epoch 315/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9425 - loss: 0.1784 - val_acc
uracy: 1.0000 - val_loss: 0.0417
Epoch 316/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1779 - val_acc
uracy: 1.0000 - val_loss: 0.0417
Epoch 317/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9310 - loss: 0.1775 - val_acc
uracy: 1.0000 - val_loss: 0.0417
Epoch 318/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1768 - val_acc
uracy: 1.0000 - val_loss: 0.0417
Epoch 319/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1765 - val_acc
uracy: 1.0000 - val_loss: 0.0416
Epoch 320/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1761 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0415
Epoch 321/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1752 - val_acc
uracy: 1.0000 - val_loss: 0.0416
Epoch 322/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.1749 - val_acc
uracy: 1.0000 - val_loss: 0.0415
Epoch 323/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.1746 - val_acc
uracy: 1.0000 - val_loss: 0.0415
Epoch 324/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9310 - loss: 0.1740 - val_acc
uracy: 1.0000 - val_loss: 0.0415
Epoch 325/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9310 - loss: 0.1735 - val_acc
uracy: 1.0000 - val_loss: 0.0413
Epoch 326/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9310 - loss: 0.1729 - val_acc
uracy: 1.0000 - val_loss: 0.0416
Epoch 327/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9310 - loss: 0.1727 - val_acc
uracy: 1.0000 - val_loss: 0.0413
Epoch 328/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9310 - loss: 0.1722 - val_acc
uracy: 1.0000 - val_loss: 0.0415
Epoch 329/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9310 - loss: 0.1717 - val_acc
uracy: 1.0000 - val_loss: 0.0413
Epoch 330/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1711 - val_acc
uracy: 1.0000 - val_loss: 0.0415
Epoch 331/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1708 - val_acc
uracy: 1.0000 - val_loss: 0.0412
Epoch 332/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1701 - val_acc
uracy: 1.0000 - val_loss: 0.0414
Epoch 333/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1700 - val_acc
uracy: 1.0000 - val_loss: 0.0412
Epoch 334/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1694 - val_acc
uracy: 1.0000 - val_loss: 0.0414
Epoch 335/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9310 - loss: 0.1688 - val_acc
uracy: 1.0000 - val_loss: 0.0409
Epoch 336/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9310 - loss: 0.1684 - val_acc
uracy: 1.0000 - val_loss: 0.0411
Epoch 337/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1681 - val_acc
uracy: 1.0000 - val_loss: 0.0409
Epoch 338/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1676 - val_acc
uracy: 1.0000 - val_loss: 0.0412
Epoch 339/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9310 - loss: 0.1670 - val_acc
uracy: 1.0000 - val_loss: 0.0411
Epoch 340/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1668 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0410
Epoch 341/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9310 - loss: 0.1662 - val_acc
uracy: 1.0000 - val_loss: 0.0409
Epoch 342/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1658 - val_acc
uracy: 1.0000 - val_loss: 0.0410
Epoch 343/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1655 - val_acc
uracy: 1.0000 - val_loss: 0.0406
Epoch 344/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1650 - val_acc
uracy: 1.0000 - val_loss: 0.0407
Epoch 345/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1646 - val_acc
uracy: 1.0000 - val_loss: 0.0407
Epoch 346/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1641 - val_acc
uracy: 1.0000 - val_loss: 0.0407
Epoch 347/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1638 - val_acc
uracy: 1.0000 - val_loss: 0.0405
Epoch 348/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1632 - val_acc
uracy: 1.0000 - val_loss: 0.0407
Epoch 349/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1631 - val_acc
uracy: 1.0000 - val_loss: 0.0405
Epoch 350/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1623 - val_acc
uracy: 1.0000 - val_loss: 0.0406
Epoch 351/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1617 - val_acc
uracy: 1.0000 - val_loss: 0.0407
Epoch 352/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9310 - loss: 0.1618 - val_acc
uracy: 1.0000 - val_loss: 0.0405
Epoch 353/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9310 - loss: 0.1612 - val_acc
uracy: 1.0000 - val_loss: 0.0403
Epoch 354/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9310 - loss: 0.1607 - val_acc
uracy: 1.0000 - val_loss: 0.0402
Epoch 355/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1603 - val_acc
uracy: 1.0000 - val_loss: 0.0402
Epoch 356/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1599 - val_acc
uracy: 1.0000 - val_loss: 0.0401
Epoch 357/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1597 - val_acc
uracy: 1.0000 - val_loss: 0.0401
Epoch 358/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1590 - val_acc
uracy: 1.0000 - val_loss: 0.0401
Epoch 359/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1587 - val_acc
uracy: 1.0000 - val_loss: 0.0399
Epoch 360/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1583 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0396
Epoch 361/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1578 - val_acc
uracy: 1.0000 - val_loss: 0.0397
Epoch 362/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9310 - loss: 0.1575 - val_acc
uracy: 1.0000 - val_loss: 0.0398
Epoch 363/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1574 - val_acc
uracy: 1.0000 - val_loss: 0.0395
Epoch 364/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9310 - loss: 0.1565 - val_acc
uracy: 1.0000 - val_loss: 0.0398
Epoch 365/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1565 - val_acc
uracy: 1.0000 - val_loss: 0.0394
Epoch 366/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1560 - val_acc
uracy: 1.0000 - val_loss: 0.0395
Epoch 367/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1559 - val_acc
uracy: 1.0000 - val_loss: 0.0395
Epoch 368/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9310 - loss: 0.1549 - val_acc
uracy: 1.0000 - val_loss: 0.0397
Epoch 369/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1549 - val_acc
uracy: 1.0000 - val_loss: 0.0395
Epoch 370/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1544 - val_acc
uracy: 1.0000 - val_loss: 0.0396
Epoch 371/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9310 - loss: 0.1537 - val_acc
uracy: 1.0000 - val_loss: 0.0396
Epoch 372/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9310 - loss: 0.1537 - val_acc
uracy: 1.0000 - val_loss: 0.0394
Epoch 373/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1530 - val_acc
uracy: 1.0000 - val_loss: 0.0394
Epoch 374/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1528 - val_acc
uracy: 1.0000 - val_loss: 0.0393
Epoch 375/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1522 - val_acc
uracy: 1.0000 - val_loss: 0.0395
Epoch 376/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1518 - val_acc
uracy: 1.0000 - val_loss: 0.0394
Epoch 377/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1516 - val_acc
uracy: 1.0000 - val_loss: 0.0392
Epoch 378/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9310 - loss: 0.1512 - val_acc
uracy: 1.0000 - val_loss: 0.0391
Epoch 379/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9310 - loss: 0.1505 - val_acc
uracy: 1.0000 - val_loss: 0.0391
Epoch 380/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1503 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0391
Epoch 381/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.9425 - loss: 0.1497 - val_acc
uracy: 1.0000 - val_loss: 0.0391
Epoch 382/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9425 - loss: 0.1498 - val_acc
uracy: 1.0000 - val_loss: 0.0390
Epoch 383/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1491 - val_acc
uracy: 1.0000 - val_loss: 0.0390
Epoch 384/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1487 - val_acc
uracy: 1.0000 - val_loss: 0.0389
Epoch 385/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9425 - loss: 0.1479 - val_acc
uracy: 1.0000 - val_loss: 0.0390
Epoch 386/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1482 - val_acc
uracy: 1.0000 - val_loss: 0.0388
Epoch 387/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9425 - loss: 0.1476 - val_acc
uracy: 1.0000 - val_loss: 0.0388
Epoch 388/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9425 - loss: 0.1470 - val_acc
uracy: 1.0000 - val_loss: 0.0388
Epoch 389/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1473 - val_acc
uracy: 1.0000 - val_loss: 0.0385
Epoch 390/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1463 - val_acc
uracy: 1.0000 - val_loss: 0.0388
Epoch 391/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1463 - val_acc
uracy: 1.0000 - val_loss: 0.0387
Epoch 392/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1455 - val_acc
uracy: 1.0000 - val_loss: 0.0387
Epoch 393/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9425 - loss: 0.1456 - val_acc
uracy: 1.0000 - val_loss: 0.0385
Epoch 394/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9425 - loss: 0.1447 - val_acc
uracy: 1.0000 - val_loss: 0.0386
Epoch 395/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9425 - loss: 0.1445 - val_acc
uracy: 1.0000 - val_loss: 0.0386
Epoch 396/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1443 - val_acc
uracy: 1.0000 - val_loss: 0.0382
Epoch 397/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1439 - val_acc
uracy: 1.0000 - val_loss: 0.0384
Epoch 398/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1437 - val_acc
uracy: 1.0000 - val_loss: 0.0383
Epoch 399/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1430 - val_acc
uracy: 1.0000 - val_loss: 0.0384
Epoch 400/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1428 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0383
Epoch 401/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1426 - val_acc
uracy: 1.0000 - val_loss: 0.0383
Epoch 402/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1418 - val_acc
uracy: 1.0000 - val_loss: 0.0383
Epoch 403/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1420 - val_acc
uracy: 1.0000 - val_loss: 0.0383
Epoch 404/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1415 - val_acc
uracy: 1.0000 - val_loss: 0.0381
Epoch 405/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1412 - val_acc
uracy: 1.0000 - val_loss: 0.0380
Epoch 406/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1408 - val_acc
uracy: 1.0000 - val_loss: 0.0381
Epoch 407/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9425 - loss: 0.1405 - val_acc
uracy: 1.0000 - val_loss: 0.0380
Epoch 408/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9425 - loss: 0.1400 - val_acc
uracy: 1.0000 - val_loss: 0.0381
Epoch 409/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1396 - val_acc
uracy: 1.0000 - val_loss: 0.0381
Epoch 410/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1395 - val_acc
uracy: 1.0000 - val_loss: 0.0379
Epoch 411/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1389 - val_acc
uracy: 1.0000 - val_loss: 0.0381
Epoch 412/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1383 - val_acc
uracy: 1.0000 - val_loss: 0.0381
Epoch 413/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1389 - val_acc
uracy: 1.0000 - val_loss: 0.0380
Epoch 414/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1378 - val_acc
uracy: 1.0000 - val_loss: 0.0380
Epoch 415/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9425 - loss: 0.1379 - val_acc
uracy: 1.0000 - val_loss: 0.0378
Epoch 416/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1368 - val_acc
uracy: 1.0000 - val_loss: 0.0379
Epoch 417/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9425 - loss: 0.1374 - val_acc
uracy: 1.0000 - val_loss: 0.0377
Epoch 418/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1367 - val_acc
uracy: 1.0000 - val_loss: 0.0378
Epoch 419/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1365 - val_acc
uracy: 1.0000 - val_loss: 0.0377
Epoch 420/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1358 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0380
Epoch 421/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9425 - loss: 0.1359 - val_acc
uracy: 1.0000 - val_loss: 0.0377
Epoch 422/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1351 - val_acc
uracy: 1.0000 - val_loss: 0.0378
Epoch 423/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1352 - val_acc
uracy: 1.0000 - val_loss: 0.0377
Epoch 424/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1345 - val_acc
uracy: 1.0000 - val_loss: 0.0379
Epoch 425/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9425 - loss: 0.1341 - val_acc
uracy: 1.0000 - val_loss: 0.0377
Epoch 426/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1342 - val_acc
uracy: 1.0000 - val_loss: 0.0376
Epoch 427/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1341 - val_acc
uracy: 1.0000 - val_loss: 0.0375
Epoch 428/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1333 - val_acc
uracy: 1.0000 - val_loss: 0.0376
Epoch 429/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1333 - val_acc
uracy: 1.0000 - val_loss: 0.0375
Epoch 430/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1326 - val_acc
uracy: 1.0000 - val_loss: 0.0376
Epoch 431/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9425 - loss: 0.1325 - val_acc
uracy: 1.0000 - val_loss: 0.0377
Epoch 432/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9425 - loss: 0.1327 - val_acc
uracy: 1.0000 - val_loss: 0.0375
Epoch 433/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.9425 - loss: 0.1311 - val_acc
uracy: 1.0000 - val_loss: 0.0374
Epoch 434/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9425 - loss: 0.1316 - val_acc
uracy: 1.0000 - val_loss: 0.0372
Epoch 435/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9425 - loss: 0.1316 - val_acc
uracy: 1.0000 - val_loss: 0.0371
Epoch 436/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1312 - val_acc
uracy: 1.0000 - val_loss: 0.0370
Epoch 437/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1305 - val_acc
uracy: 1.0000 - val_loss: 0.0372
Epoch 438/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1304 - val_acc
uracy: 1.0000 - val_loss: 0.0371
Epoch 439/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1303 - val_acc
uracy: 1.0000 - val_loss: 0.0371
Epoch 440/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9425 - loss: 0.1300 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0370
Epoch 441/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1290 - val_acc
uracy: 1.0000 - val_loss: 0.0370
Epoch 442/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1292 - val_acc
uracy: 1.0000 - val_loss: 0.0370
Epoch 443/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9425 - loss: 0.1295 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 444/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1281 - val_acc
uracy: 1.0000 - val_loss: 0.0369
Epoch 445/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1284 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 446/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1283 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 447/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1280 - val_acc
uracy: 1.0000 - val_loss: 0.0367
Epoch 448/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9540 - loss: 0.1272 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 449/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1277 - val_acc
uracy: 1.0000 - val_loss: 0.0366
Epoch 450/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1268 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 451/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1268 - val_acc
uracy: 1.0000 - val_loss: 0.0366
Epoch 452/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1262 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 453/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1265 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 454/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1256 - val_acc
uracy: 1.0000 - val_loss: 0.0367
Epoch 455/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1255 - val_acc
uracy: 1.0000 - val_loss: 0.0366
Epoch 456/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9540 - loss: 0.1248 - val_acc
uracy: 1.0000 - val_loss: 0.0366
Epoch 457/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1254 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 458/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1243 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 459/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1244 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 460/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9540 - loss: 0.1242 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0365
Epoch 461/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1234 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 462/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1233 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 463/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1234 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 464/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9540 - loss: 0.1227 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 465/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9540 - loss: 0.1229 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 466/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9540 - loss: 0.1220 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 467/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9540 - loss: 0.1221 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 468/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9540 - loss: 0.1212 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 469/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9540 - loss: 0.1216 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 470/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.9540 - loss: 0.1210 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 471/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1203 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 472/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9540 - loss: 0.1209 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 473/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1200 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 474/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1204 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 475/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1195 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 476/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9540 - loss: 0.1192 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 477/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9540 - loss: 0.1191 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 478/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1187 - val_acc
uracy: 1.0000 - val_loss: 0.0362
Epoch 479/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1184 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 480/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1184 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0360
Epoch 481/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1179 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 482/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1181 - val_acc
uracy: 1.0000 - val_loss: 0.0360
Epoch 483/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9540 - loss: 0.1170 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 484/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9540 - loss: 0.1175 - val_acc
uracy: 1.0000 - val_loss: 0.0362
Epoch 485/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9540 - loss: 0.1168 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 486/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9540 - loss: 0.1170 - val_acc
uracy: 1.0000 - val_loss: 0.0364
Epoch 487/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1160 - val_acc
uracy: 1.0000 - val_loss: 0.0363
Epoch 488/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1157 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 489/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1161 - val_acc
uracy: 1.0000 - val_loss: 0.0365
Epoch 490/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1151 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 491/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1145 - val_acc
uracy: 1.0000 - val_loss: 0.0366
Epoch 492/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9540 - loss: 0.1145 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 493/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1145 - val_acc
uracy: 1.0000 - val_loss: 0.0369
Epoch 494/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1137 - val_acc
uracy: 1.0000 - val_loss: 0.0369
Epoch 495/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1141 - val_acc
uracy: 1.0000 - val_loss: 0.0371
Epoch 496/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.9540 - loss: 0.1130 - val_acc
uracy: 1.0000 - val_loss: 0.0372
Epoch 497/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1128 - val_acc
uracy: 1.0000 - val_loss: 0.0371
Epoch 498/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9540 - loss: 0.1125 - val_acc
uracy: 1.0000 - val_loss: 0.0371
Epoch 499/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9540 - loss: 0.1125 - val_acc
uracy: 1.0000 - val_loss: 0.0368
Epoch 500/500
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9540 - loss: 0.1120 - val_acc
```

```
uracy: 1.0000 - val_loss: 0.0372

¡Entrenamiento completado!
Número de épocas ejecutadas: 500
```

Una vez entrenado el modelo lo aplicaremos al conjunto de prueba. Para estudiar los resultados, mostraremos la precisión del modelo y la matriz de confusión con el objetivo de evaluar si hay un patrón.

In [28]:
```
from sklearn.metrics import (accuracy_score, classification_report, confusion_ma

# Predicciones de la red neuronal
y_prob = model.predict(X_test_scaled)

# Convertimos probabilidades en clases: de esta forma las probabilidades mayores
y_hat_test = (y_prob >= 0.7).astype(int).ravel()

# Accuracy
print("Red Neuronal - Accuracy:", accuracy_score(y_test, y_hat_test))

# Informe de clasificación
print("\nInforme de clasificación:")

print(classification_report(y_test, y_hat_test, target_names=['Psychic', 'Fighti

# Matriz de confusión
print("\nMatriz de confusión:")
print(confusion_matrix(y_test, y_hat_test))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 135ms/step
Red Neuronal - Accuracy: 0.8928571428571429

Informe de clasificación:
              precision    recall  f1-score   support

     Psychic       0.89      0.80      0.84        10
    Fighting       0.89      0.94      0.92        18

    accuracy                           0.89        28
   macro avg       0.89      0.87      0.88        28
weighted avg       0.89      0.89      0.89        28


Matriz de confusión:
[[ 8  2]
 [ 1 17]]
```

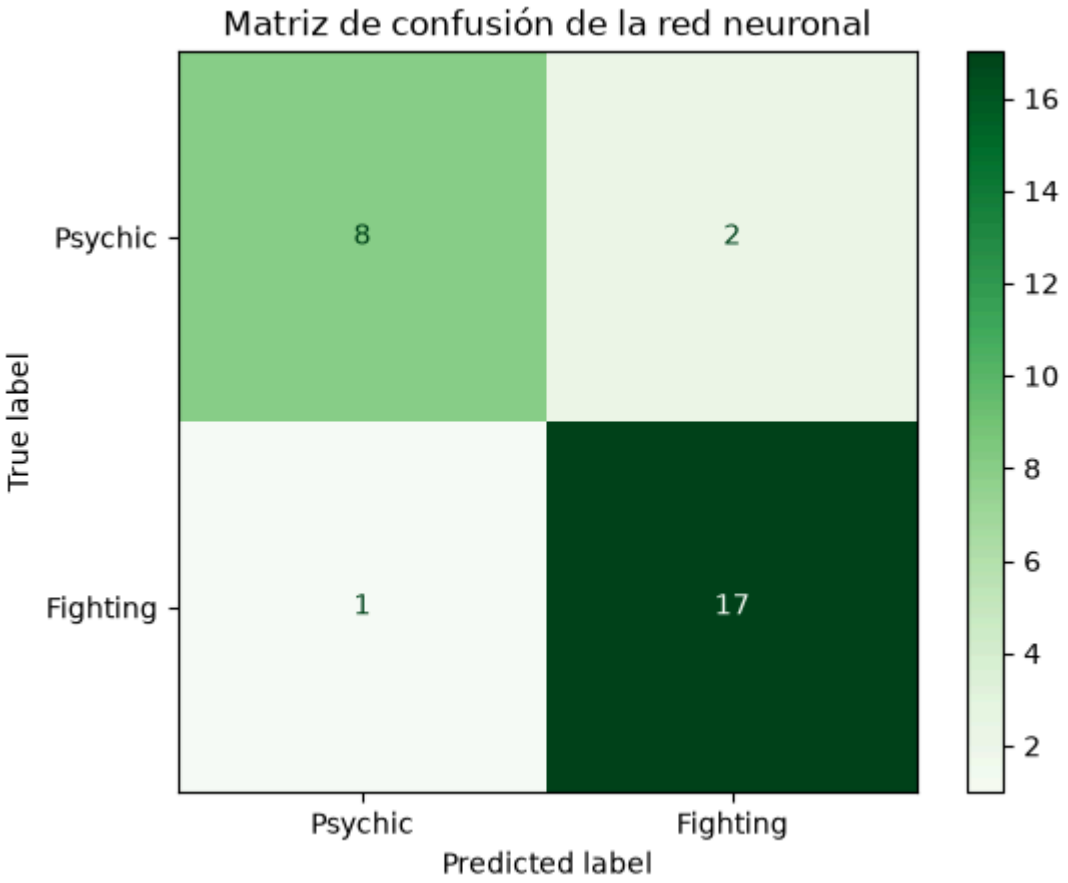
In [29]:
```
from sklearn.metrics import ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_hat_test)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Psychic', 'F

disp.plot(cmap='Greens')
```

```
plt.title('Matriz de confusión de la red neuronal')
plt.show()
```



Hemos obtenido un valor de precisión considerablemente alto. Además, la matriz de confusión nos corrobora el correcto funcionamiento del modelo, el cual clasifica especialmente bien los tipo lucha.

Por lo tanto, efectivamente: ¡Hay patrones en algunos tipos de pokémon!

Cabe destacar el hecho de que es probable que se pueda obtener un mejor resultado si restringimos nuestra variable X a solo aquellas estadísticas que estén más diferenciadas entre si.

También podría ser interesante comprobar más estereotipos de cada tipo: si los voladores tienen mucha velocidad, si los tipo dragón tienen estadísiticas más altas que el resto, si los tipos roca y acero tienen más defensa que los demás... Hay muchos estereotipos que se tienen por ciertos a la hora de jugar que podrían ser estudiados.

Ahora, a modo de comprobación, representaremos las curvas de evolución de la precisión y la pérdida a lo largo de las épocas. Si no hemos sobreentrenado el modelo, la curva de pérdida de validación debería mantenerse decreciente.

In [30]:
```
# Extraemos los datos del historial
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)


# Gráfica de Accuracy
plt.figure(figsize=(8, 5))

plt.plot(epochs, acc, label='Entrenamiento')

plt.plot(epochs, val_acc, label='Validación')

plt.title('Precisión de la red neuronal')
plt.xlabel('Épocas')
plt.ylabel('Accuracy')

plt.legend()
plt.grid(True)
plt.show()

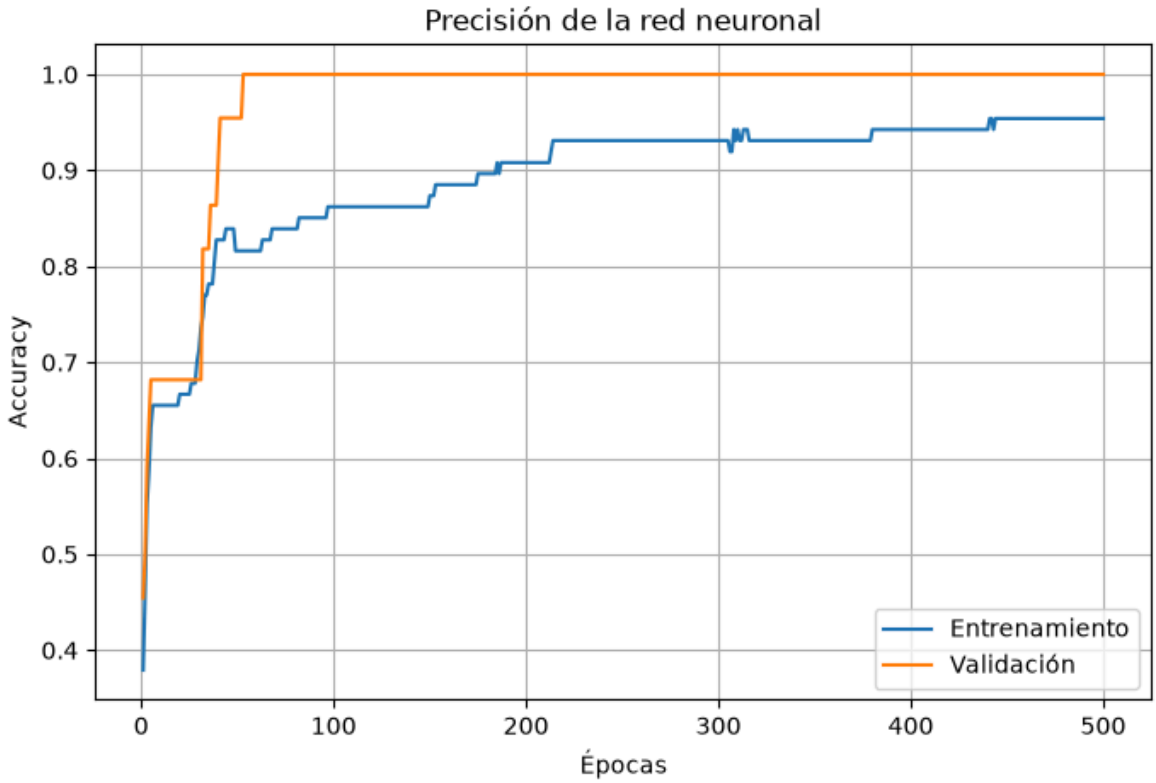

# Gráfica de Loss
plt.figure(figsize=(8, 5))

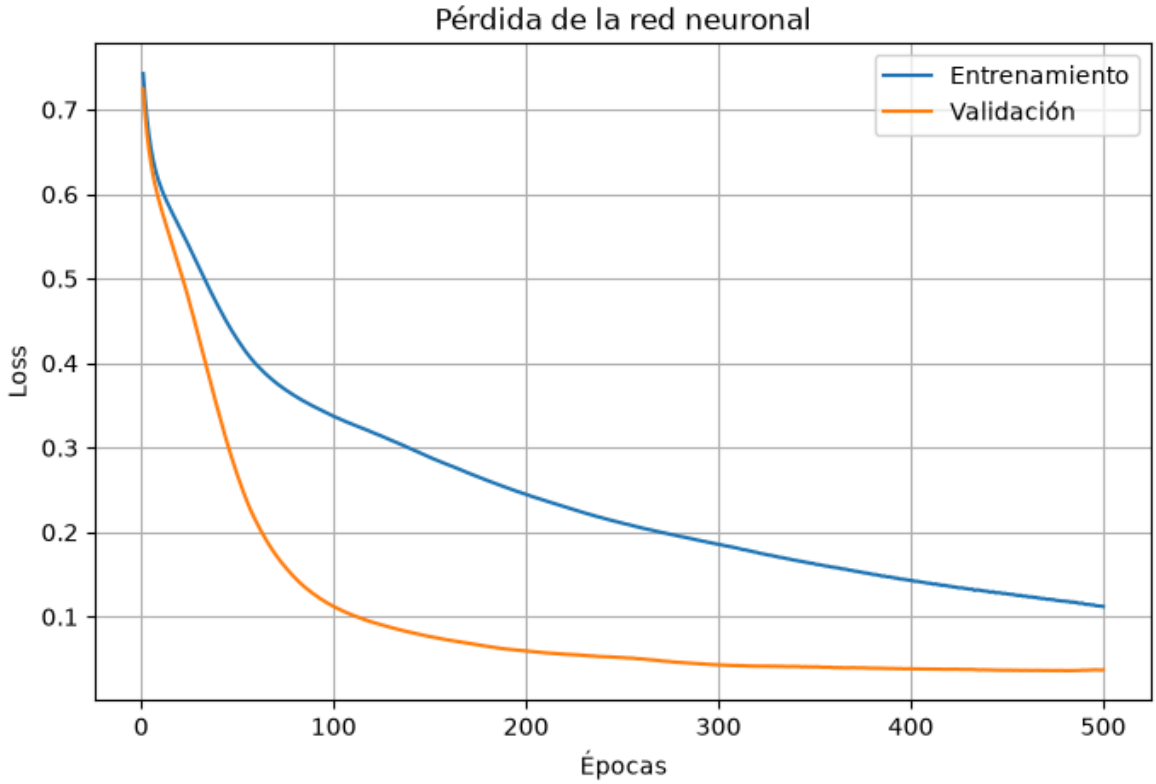
plt.plot(epochs, loss, label='Entrenamiento')

plt.plot(epochs, val_loss, label='Validación')

plt.title('Pérdida de la red neuronal')
plt.xlabel('Épocas')
plt.ylabel('Loss')

plt.legend()
plt.grid(True)
plt.show()
```





Vemos que la curva de pérdida efectivamente acaba en un mínimo: no hay sobreentrenamiento.

Parte 3: Siempre buscamos los legendarios

Al jugar a la franquicia, lo habitual es prestar especial atención a los pokémon *legendarios*, ya que se presupone que poseen estadísticas muy superiores al resto y suelen protagonizar las carátulas de los videojuegos. Aprovechando que el conjunto de datos dispone de esta clasificación, analizaremos si los ejemplares con mejores estadísticas pertenecen exclusivamente a este grupo o si existen excepciones relevantes.

Estas excepciones pueden surgir porque una clasificación estrictamente binaria simplifica en exceso la realidad del juego, donde coexisten otras categorías con estadísticas muy altas, como los pokémon *singulares* o los *pseudolegendarios*.

Con todo, utilizaremos esta etiqueta para entrenar una nueva red neuronal con una arquitectura alternativa y evaluar su capacidad de clasificación.

In [31]:
```
#Vamos a volver a cargar el dataset original, ya que lo habíamos filtrado.
df = pd.read_csv('Pokemon.csv')

print('\n' + '=' * 50)
display(df.head())
print('\n' + '=' * 50)

df['Legendary'].unique()
```

```
==================================================
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Genera
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	

```
==================================================
```

Out[31]:
```
array([False,  True])
```

In [32]:
```
#Vamos a poner los valores de la columna 'Legendary' como 0 y 1, en lugar de Tru

df['Legendary'] = df['Legendary'].astype(int)
```

In [33]:
```
#Vamos a escoger ahora como X ya no solo las estadísticas, sino también la estad

X = df[estadísticas + ['Total']].values
Y = df['Legendary'].values

# Primera división: entrenamiento y prueba
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.20, random

# División del entrenamiento para obtener validación
X_train, X_val, Y_train, Y_val = train_test_split(X_train, Y_train, test_size=0.

# Escalado de las variables
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)


print('\n' + '=' * 50)


print('\nDatos de entrenamiento:', X_train_scaled.shape)
print('Datos de validación:', X_val_scaled.shape)
print('Datos de prueba:', X_test_scaled.shape)

print('\n' + '=' * 50)
```

```
==================================================

Datos de entrenamiento: (512, 7)
Datos de validación: (128, 7)
Datos de prueba: (160, 7)

==================================================
```

Creamos ahora el modelo que vamos a seguir. Esta vez vamos a hacer la siguiente red neuronal, probando a cambiar el tipo de activación:

1. Una neurona de input tipo RELU
2. Segunda cada de 64 neuronas tipo sigmoide
3. Una tercera etapa igual que la segunda
4. Una última de salida tipo RELU

In [34]:
```
np.random.seed(42)
tf.random.set_seed(42)

# Creación de la red neuronal
model = keras.Sequential([layers.Dense(1, activation='relu', input_shape=(X_trai

# Compilación del modelo
sgd = SGD(learning_rate=0.01)

model.compile(optimizer=sgd, loss='binary_crossentropy', metrics=['accuracy'])


# Resumen de la arquitectura
model.summary()
```

```
c:\Users\david\anaconda3\envs\tensorflow_env\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 1)	8
dense_5 (Dense)	(None, 64)	128
dense_6 (Dense)	(None, 64)	4,160
dense_7 (Dense)	(None, 1)	65

Total params: 4,361 (17.04 KB)

Trainable params: 4,361 (17.04 KB)

Non-trainable params: 0 (0.00 B)

In [35]:
```
early_stop = EarlyStopping(monitor='val_loss', patience=40, restore_best_weights

# Entrenamiento
print("Entrenando la red neuronal...")

history = model.fit(X_train_scaled, Y_train, epochs=500, batch_size=10, validati

print('\n¡Entrenamiento completado!')

print('\nNúmero de épocas ejecutadas:', len(history.history['loss']))
```

```
Entrenando la red neuronal...
Epoch 1/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.9180 - loss: 1.3222 - val_a
ccuracy: 0.9219 - val_loss: 1.2592
Epoch 2/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 3/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 4/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 5/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 6/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 7/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 8/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 9/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 10/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 11/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 12/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 13/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 14/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 15/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 16/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 17/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 18/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 19/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 20/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
```

```
curacy: 0.9219 - val_loss: 1.2592
Epoch 21/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 22/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 23/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 24/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 25/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 26/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 27/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 28/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 29/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 30/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 31/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 32/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 33/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 34/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 35/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 36/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 37/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 38/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 39/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592
Epoch 40/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
```

```
curacy: 0.9219 - val_loss: 1.2592
Epoch 41/500
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9180 - loss: 1.3222 - val_ac
curacy: 0.9219 - val_loss: 1.2592

¡Entrenamiento completado!

Número de épocas ejecutadas: 41
```

Vemos que se ejecutan pocas épocas por el hecho de que hay un porcentaje muy pequeño de legendarios, por lo que llegamos rápidamente al sobreentrenamiento.

In [36]:
```
# Predicciones de la red neuronal
Y_prob = model.predict(X_test_scaled)

# Convertimos probabilidades en clases: de esta forma las probabilidades mayores
Y_hat_test = (Y_prob >= 0.7).astype(int).ravel()
# Accuracy
print("Red Neuronal - Accuracy:", accuracy_score(Y_test, Y_hat_test))

# Informe de clasificación
print("\nInforme de clasificación:")

print(classification_report(Y_test, Y_hat_test, target_names=['No legendario', '

# Matriz de confusión
print("\nMatriz de confusión:")
print(confusion_matrix(Y_test, Y_hat_test))

cm = confusion_matrix(Y_test, Y_hat_test)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['No legendari

disp.plot(cmap='Greens')

plt.title('Matriz de confusión de la red neuronal')
plt.show()
```

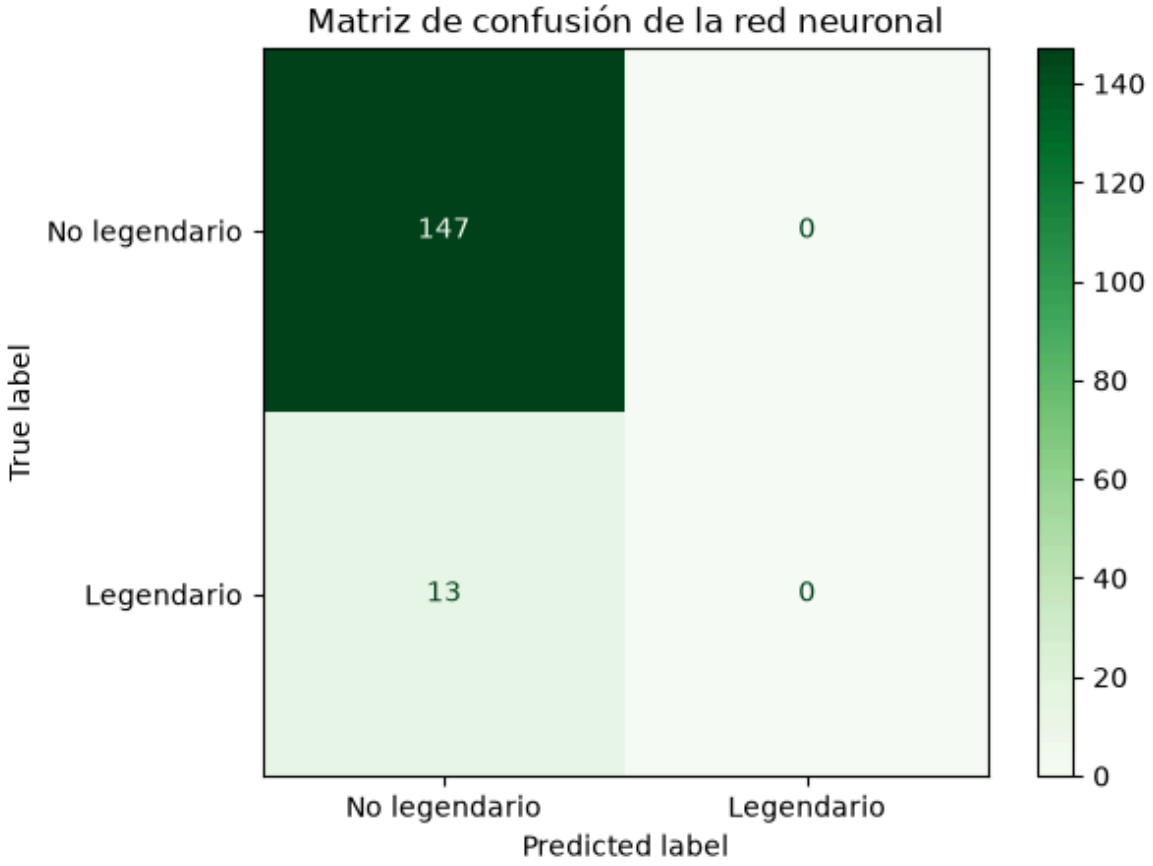
```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
Red Neuronal - Accuracy: 0.91875

Informe de clasificación:
              precision    recall  f1-score   support

No legendario       0.92      1.00      0.96       147
   Legendario       0.00      0.00      0.00        13

    accuracy                           0.92       160
   macro avg       0.46      0.50      0.48       160
weighted avg       0.84      0.92      0.88       160


Matriz de confusión:
[[147   0]
 [ 13   0]]
```



Aunque el modelo registra una precisión muy alta, su funcionamiento no es el adecuado. El motivo principal es la escasez de pokémon legendarios en el conjunto de datos. Al ser tan poco frecuentes, es probable que en las particiones de los datos apenas se incluyan ejemplos de esta clase, afectando negativamente al aprendizaje del modelo.

Conclusión

Los resultados reflejan que *Game Freak* mantiene cierta coherencia en los parámetros de diseño de los tipos: los atributos de un pokémon parecen influir directamente en sus estadísticas base, un fenómeno especialmente visible en los tipos psíquico y lucha. Aunque a nivel general se asume que no existe una relación directa entre ambas variables, el análisis detallado demuestra que sí hay un impacto significativo.

Asimismo, la condición de "legendario" altera notablemente el potencial del pokémon. No obstante, el modelo podría presentar un sesgo en sus predicciones debido a la baja representatividad de pokémon legendarios en el dataset.

